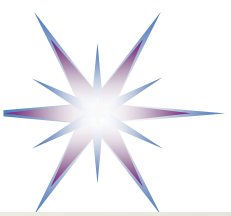


Agentic Systems and Agentic Enterprise

Integrated Agentic Systems Reference Architecture and Agentic Development Environment

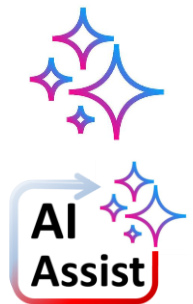
Brief introduction to initiate thinking

Yuri Demchenko
UvA, NTUU “Igor Sikorsky KPI”



Outline and Concepts – Planned structure

- Background and playground: Human – AI interaction and Knowledge Presentation, Representation, Perception
- Software Complexity and Engineering Governance (SCEG) model
 - Competence framework definition and role profiles
- Agentic AI and Agentic Systems: Agents, Models, Architecture, Components
- Integrated Agentic Systems Reference Architecture (IASRA) and an Agentic Development Environment
 - Agentic systems design patterns
- Agentic Knowledge Architecture (AKA) and Team Agentic Knowledge Service (TAKS)
- Additional Information
 - Summary Anthropic article “2026 Agentic Coding Trends Report. How coding agents are reshaping software development”
 - Transformative effect for Applications and Services companies
 - Knowledge-Complexity and Epistemic-Governance Model of LLM-Assisted Research (KCEG)



Sign indicating Generated with GenAI/ChatGPT

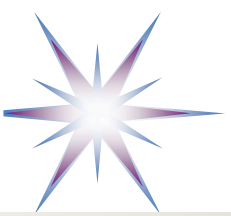
or Assisted with ChatGPT (AGI)

Disclaimer

This research and development is based on and built around the concept of Knowledge.

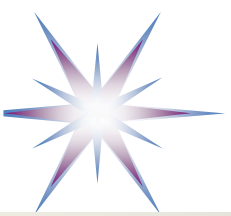
- Knowledge doesn't have measurable properties similar to (research) experiments.
- Knowledge has a variety of presentation and expression forms
- But knowledge can be validated with epistemic correctness – attributed only to humans/researchers and dependent on their personal epistemic model

Consequently, the presented information (working knowledge) may have variable form; however, it clearly represents the author's epistemic model of the research topic



Setting up a playground

- Conceptual chain for Human - AI interaction for HGLEP/KCEG
- Human – AI interface
- Knowledge Complexity: Presentation – Representation - Perception



Conceptual chain for Human - AI interaction for HGLEP/KCEG

1. Knowledge source

- paper, dataset, prior dialogue, experiment, model, human expertise

2. Knowledge presentation

- text, diagram, table, formula, code, LLM response

3. Knowledge representation

- concepts, relations, ontology, claims, evidence, assumptions, methods

4. Knowledge perception / reconstruction

- human or AI agent interprets and reconstructs the meaning

5. Knowledge assessment

- KCEG evaluates level, completeness, uncertainty, reliability, novelty

6. Internalisation / agent memory update

- human updates personal epistemic model
- AI agent updates context, memory, graph, or retrieval state

7. Knowledge action

- explanation, critique, decision, experiment, paper writing, new prompt

8. Regeneration

- human or AI generates a new presentation

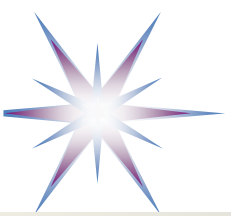
Basis for a generative loop:

presentation → perception → representation → assessment → memory → action → new presentation

Knowledge is an actional element of the human cognitive composition

- Knowledge structure/direct actions
- Built-in/constructed/re-constructed knowledge structure defines learning efficiency and effective curricula

KCEG – Knowledge Complexity and Epistemic Governance
HGLEP – Human – Governed LLM Epistemic Knowledge
Production



Human - AI Epistemic Interaction Model

Human agent:

Personal epistemic model

- semantic memory
- methodological knowledge
- tacit expertise
- epistemic judgment
- goals / intentions

AI agent:

Agent knowledge state

- context window
- retrieved knowledge
- latent model representation (ML internal)
- symbolic/graph memory
- tool outputs
- generated presentations

Interaction process:

Human intention



Prompt / instruction presentation



AI internal representation



Generated knowledge presentation



Human perception and reconstruction/representation



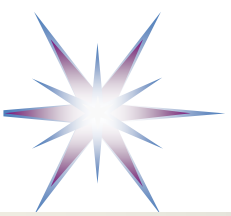
KCEG assessment (by human/researcher)



HGLEP-guided refinement



Updated human and AI knowledge states



Knowledge Presentation – Representation - Perception

Available/Received Knowledge

Knowledge Presentation

1. Vocabulary/Terminology complexity (KC1)
2. Semantic complexity (KC2)
3. Ontological complexity (KC3)
4. Structural complexity (KC4)
5. Methodological complexity (KC5)
6. Epistemic complexity/validity (KC6)

Knowledge inferential/LLM, Internal representation

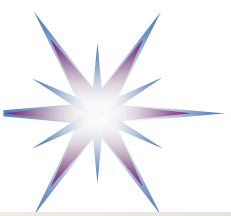
Knowledge Representation

1. Vocabulary / terminology (K1)
2. Syntax / expression form (K2)
3. Factual / propositional layer (K3)
4. Semantic layer (K4)
5. Ontological layer (K5)
6. Structural / graph layer (K6)
7. Inferential layer (K7)
8. Methodological layer (K8)
9. Epistemic / research-frontier layer (K9)

Perception via Information, Epistemic Validation, Internalisation

Knowledge perception

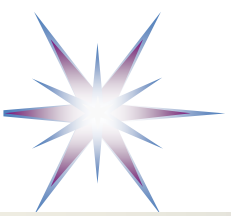
- If a human doesn't have a full Kx or KCx model of KCEG, they may treat received/observed information structured wrongly or a kind of imaginary mixed
- This is the case for children
 - Can we control and speed up development?
 - Can we do the same for Doctoral (PhD) Students?
- Children imaginary treating objects and animals with human features
 - Hypothetically because of KCEG
 - Agent-centered perception model by children
- TODO: Adding Pragmatics to Semantics Complexity level as related to language understanding by humans



KCEG Perception by Expert and Novice

Mechanism: Perception is holistic, understanding is selective reconstruction

- **1. Real object/event/system**
The full **THING EXISTS** with many properties: visible, functional, structural, causal, methodological, historical, epistemic.
- **2. Sensory/perceptual intake**
The person receives **SIGNALS**: visual form, words, diagrams, behaviour, motion, numbers, names, artefacts.
 - -----
- **3. Cognitive reconstruction**
The **MIND MAPS** signals into existing *schemas*: vocabulary, concepts, relations, structures, methods, causal models, epistemic status.
- **4. Expressed understanding**
The person explains, draws, predicts, classifies, evaluates, or acts.
- **Hypothesis: Gap between layers 2 and 3/4.**
 - The person may “see” the whole event, but their **model of what is significant** is constrained by their available cognitive schemas.
 - This is close to the expert-novice literature review: experts detect meaningful patterns that novices overlook, and experts organise knowledge around deep principles rather than surface attributes.



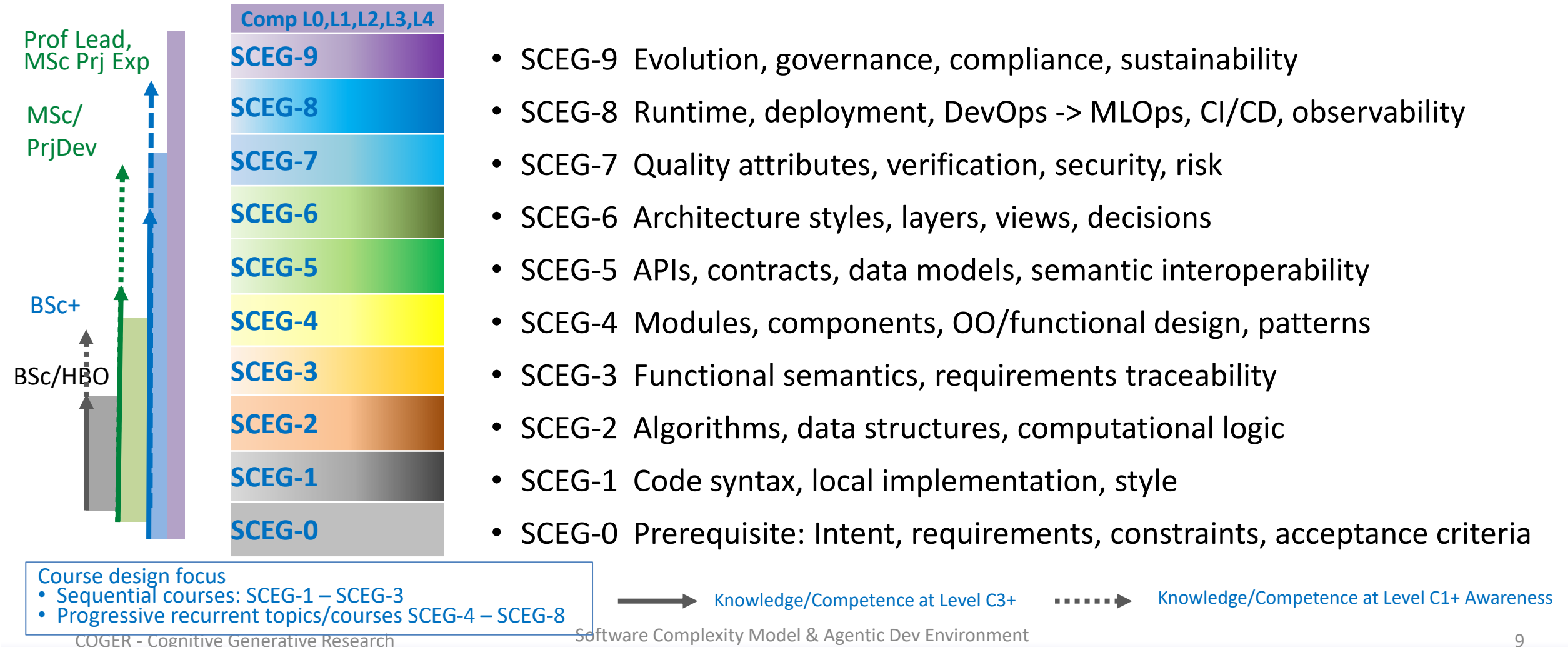
Software Complexity and Engineering Governance (SCEG) model

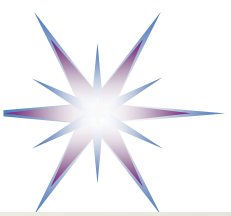
- Software Complexity and Engineering Governance (SCEG) model as basis for software structuring and architecture definition
 - Extending SCEG with a competence framework and governed agentic coding
 - Example definition and role profiles



Software Complexity Model (SCEG) as a basis for System and Software Engineering Curriculum with an indication of Competence Levels

code → modules → data models → APIs → architecture → quality attributes → runtime operation → lifecycle governance

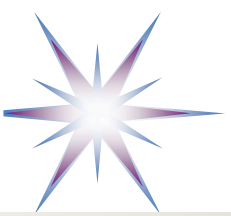




SCEG Competence and Role Profiling Model — SCEG-CRP

SCEG levels describe increasing forms of software-system complexity, but practitioner profiles need not progress through them sequentially. Professional competence is represented by the proficiency demonstrated independently at each SCEG level and by the ability to integrate related levels when performing a role.

- Foundational components
 - SCEG software complexity model: S0–S9
 - Competence scale: C1–C5
 - Agentic work mode and governed agentic coding: A0 – A4
- Practical implementation
 - Evidence record for each assessed cell
 - Individual competence profile
 - Target role profile
 - Team coverage profile
 - Course or development transformation profile



Competence Levels for each of SCEG Level

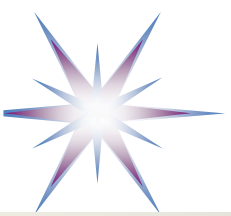
Competence	Proposed name	Characteristic capability
C1	Aware	Recognises concepts, terminology, artefacts and common problems
C2	Contributing	Performs bounded tasks using established guidance, examples and review
C3	Independent	Completes, explains, verifies and corrects normal professional work independently
C4	Owning and integrating	Owens a subsystem or practice, handles difficult cases, integrates work across boundaries and guides others
C5	Evolving and governing	Defines approaches, evaluates systemic trade-offs, establishes organisational practice and evolves the capability

- Competences expanding to practical and expert professional work
- Professional profiles/roles may not require a full competence level for each SCEG level
- Target MSc competence level C3
 - To be a subject of assessment

Competence Assessment Dimensions

- Understanding -- Performance -- Verification -- Autonomy and responsibility -- Transfer and improvement

For AI-assisted development, **tool operation and prompting are not sufficient evidence of competence**. Competence requires understanding, verification, correction and responsibility for the resulting software.



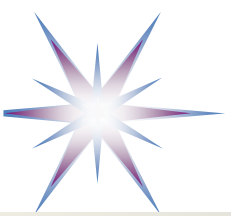
Example Competence profiles for typical roles

Role	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9
Application developer	C (C2)	O	I	I	I	C/I	C	I	C	A (C1)
Backend/service engineer	C (C2)	O	I	O	O	O	I	I	I	C
DevOps/platform engineer	C (C2)	I	C	C	C	I	I	O (C4)	O/G	I
SRE	C (C2)	I	I	I	C	I	I	O (C4)/ G (C5)	O/G	I/O
Test/quality engineer	C (C2)/ I (C3)	C	C	O	I	I	I	O (C4)/ G (C5)	I	C/I
Software architect	O (C4)	I	C/I	O	O	O	G	O	I/O	O
Technical lead	I/O	O	I	O	O	O	O	O	I/O	I/O
Engineering manager	O	C	A (C1)/ C (C2)	I	I	I	O	O	I	O/G

- Competences expanding to practical and expert professional work
- Education and degree provide background systemic structured knowledge at level C1-C3: awareness, contributing, independent
- Master thesis should bring to C4 – Owning/integrating

- **Legend:**

A = C1 awareness, C = C2 contributing, I = C3 independent, O = C4 owning/integrating, G = C5 governing/evolving, – = normally not central.



SCEG and Competence Model for Agentic SSE

Layer 1 — SCEG complexity model

SCEG continues to describe the objects and relationships whose complexity must be mastered:

- requirements and intent;
- local code and algorithms;
- functional semantics;
- modules and components;
- APIs and data models;
- architecture;
- quality and risk;
- deployment and operations;
- evolution and governance.
- This layer should change slowly and only when the conceptual structure of software engineering changes—not when a new coding product is released.

Layer 2 — SCEG competence profile

The C1–C5 competence scale continues to describe human capability:

- C1 Aware;
- C2 Contributing;

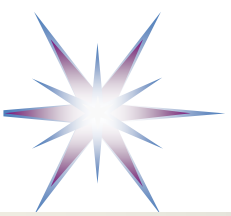
- C3 Independent;
- C4 Owning and integrating;
- C5 Evolving and governing.

Competence descriptions should become **tool-neutral**.

Layer 3 — Agentic work mode

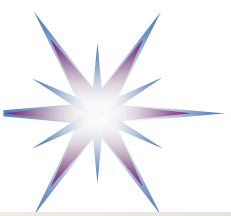
Each competence cell can optionally carry an agentic-work qualifier:

- A0 Direct
 - The practitioner performs the activity directly, without substantive AI assistance
- A1 AI-assisted
 - AI supports explanation, completion, analysis or limited generation
- A2 Agent-delegated
 - A bounded engineering task is delegated to an agent and reviewed by the practitioner
- A3 Agent-orchestrated
 - The practitioner coordinates long-running or multiple agents and integrates their outputs
- A4 Agent-governed environment
 - The practitioner designs organisational guardrails, platforms, policies and assurance systems for agentic work



Example Profiles

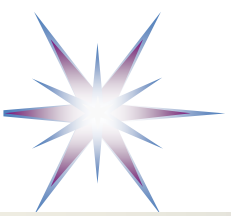
- Developer profile S5-C3-A2
 - Independently competent in APIs and data models, normally working by delegating bounded API tasks to an agent and verifying the result.
- Architect profile S6-C4-A3
 - Owns and integrates architectural work while coordinating several agent activities.
- Platform leader profile S8-C5-A4
 - Defines and governs the organisational environment through which agents produce, test and deploy software.



Evidence-based assessment & Specific for AI-assisted work

Evidence category	Examples
Produced artefacts	Code, API specifications, schemas, architecture views, tests, pipelines, dashboards
Engineering decisions	ADRs, trade-off analyses, risk assessments, migration decisions
Verification evidence	Reviews, test results, incident diagnosis, conformance checks
Operational evidence	Deployments, rollback execution, monitoring improvements, incident reports
Explanation	Design walkthrough, oral defence, review discussion, technical documentation
Improvement and influence	Coaching, reusable patterns, standards, process or platform improvements

- Each competence rating should require evidence appropriate to the SCEG domain.
- For AI-assisted work, additional evidence should show that the practitioner:
 - identified assumptions in the generated result;
 - checked it against requirements and architecture;
 - tested or analysed its behaviour;
 - corrected significant deficiencies;
 - can explain why the final result is acceptable;
 - assumes professional responsibility for it.
- A chat transcript may support the assessment, but it does not by itself demonstrate competence.



Agentic System Reference Architecture: Structural Design and Agentic Environment

Agent types

- Domain Agent
- Orchestrator Agent
- Retrieval/Context Agent
- Tool/Actions Agent
- Monitoring Agent
- Verification Agent
- Human-escalation Agent

Agents Configuration and Setup

- Agent name
- Tools
- Memory
- Permissions
- Guardrails
- Output

Agentic Knowledge Architecture (AKA) and ADE (Agentic Development Environment)

- Documents, policies, logs, metrics, ontologies, APIs, rules, memory
- Process models
- Team Knowledge agent
- Shared team knowledge
- Specialised agents

OE. Operational Agentic Environment

OE.1 Inputs: humans, events, and tasks

The system receives work from two major sources.

OE.2 Supervisor or orchestrator agent

OE.3 Task state and workflow engine

OE.4. Enterprise tools and action systems

Governance and assurance

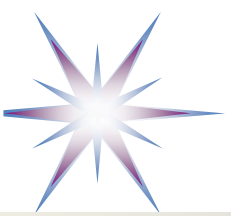
G.1 Human-in-the-loop.

G.2 Role-based access and security

G.3 Guardrails and policy

G.4 Audit, tracing, and observability

G.5 Quality, risk, and compliance



Model Types and Agentic Development Environment (ADE)

Model types

- LLM
 - Reasoning, planning, dialogue, tool selection
- Embedding model
 - Semantic search, RAG, memory retrieval
- Reranker
 - Improves retrieved context quality
- Classifier
 - Risk classification, routing, anomaly category
- Forecasting model
 - SLA prediction, demand forecast, failure prediction
- Vision/audio model
 - Document, image, speech, sensor analysis

Deployment flow

Model development



Model evaluation



Model registry / versioning



Containerisation or managed endpoint



Inference serving



API gateway / model gateway



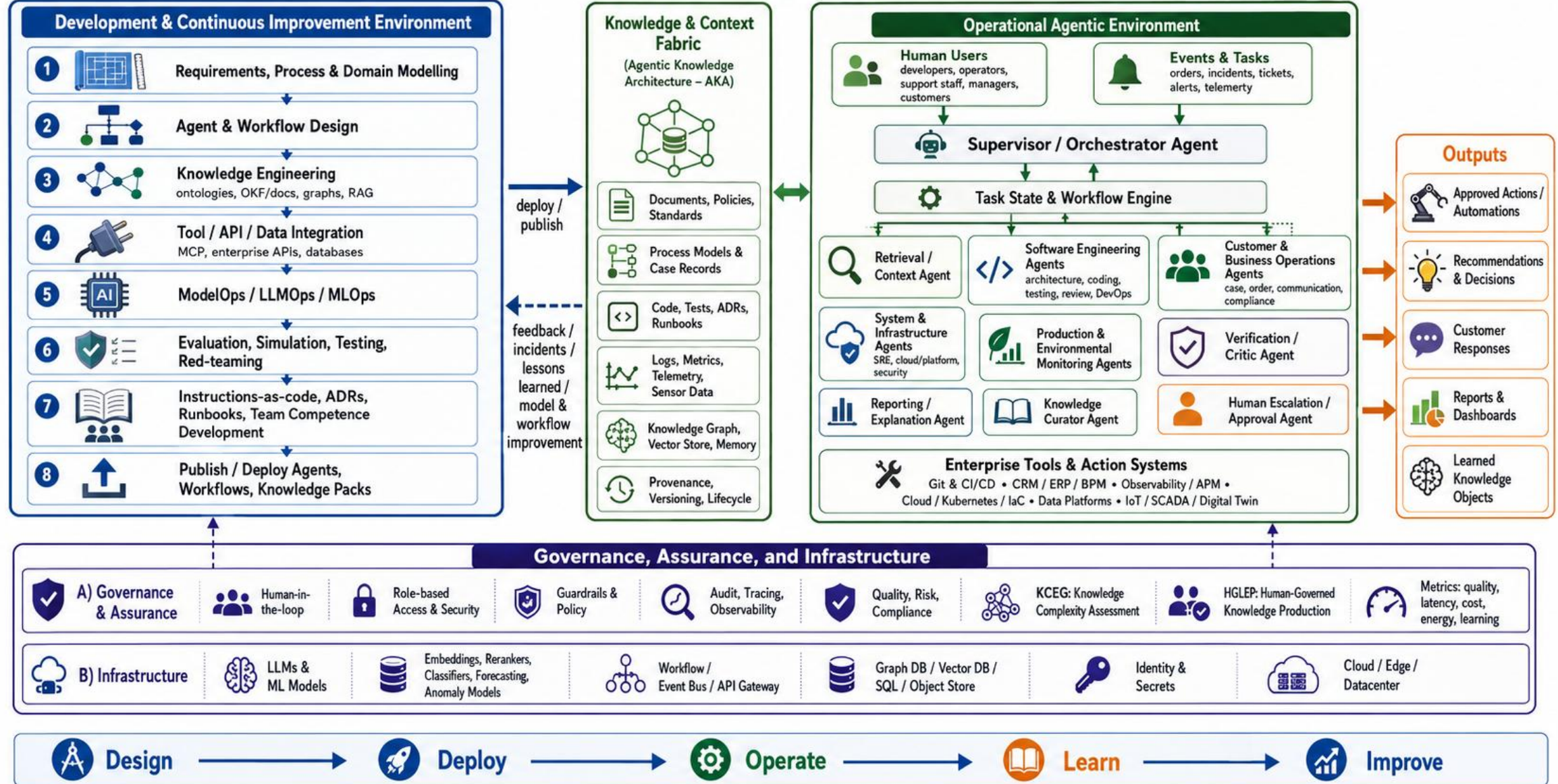
Agent calls model endpoint

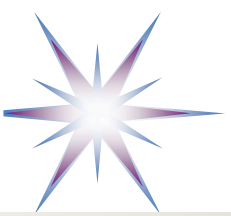


Monitoring: latency, cost, quality, drift, safety

Integrated Agentic Systems Reference Architecture (IASRA)

Development and operational agentic environment for software engineering, business operations, customer interaction, system & infrastructure engineering, production, and environmental monitoring

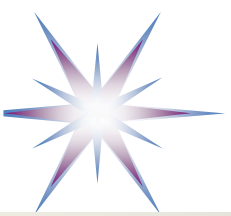




Reference Case: Integrated Agentic Systems Reference Architecture (IASRA)

The architecture consists of four major parts

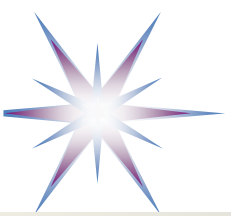
- D. Development and Continuous Improvement Environment
- K. Knowledge and Context Fabric
- O. Operational Agentic Environment
- G. Governance, Assurance, and Infrastructure



D. Development and Continuous Improvement Environment (activities)

D. Development and Continuous Improvement Environment (activities)

- D.1 Requirements, process, and domain modelling
The starting point is not the LLM or the agent.
- D.2 Agent and workflow design
The agent has a defined role, authority, tools, context, state, and operational responsibility.
- D.3 Knowledge engineering
Knowledge engineering prepares the information and knowledge that agents require.
This activity connects the Agentic Knowledge Architecture, AKA, to the operational environment.
- D.4 Tool, API, and data integration
Agents require controlled access to external capabilities.
Protocols such as MCP may be used to expose resources and tools, while conventional API gateways and identity mechanisms still control access.
- D.5 ModelOps, LLMOps, and MLOps
The system may contain several models, not only one LLM.
This activity manages model selection, deployment, evaluation, versioning, performance, cost, and replacement.
- D.6 Evaluation, simulation, testing, and red-teaming
Before deployment, agents and workflows should be evaluated
For durable systems, evaluation should cover not only the quality of an LLM answer but also the correctness of the complete workflow and its consequences.
- D.7 Instructions-as-code, ADRs, runbooks, and competence development
This component captures the operational and engineering knowledge needed to maintain
- D.8 Publication and deployment
Validated artifacts are packaged and deployed

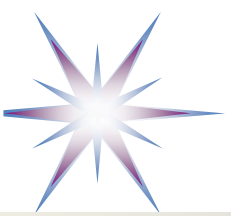


K. Knowledge and Context Fabric

K. Knowledge and Context Fabric

Connects development and operations, providing a common knowledge environment for humans, agents, workflows, and models. It contains several knowledge categories.

- K.1 Documents, policies, and standards
- K.2 Process models and case records
- K.3 Code, tests, ADRs, and runbooks
For software and infrastructure engineering, knowledge also exists in operational artifacts. These artifacts represent procedural, structural, and methodological knowledge.
- K.4 Logs, metrics, telemetry, and sensor data
This is the dynamic operational context.
It allows agents to reason about the current state rather than only static documents.
- K.5 Knowledge graph, vector store, and memory
Different representations serve different purposes
- K.6 Provenance, versioning, and lifecycle
Every significant knowledge object should include.
This turns a collection of documents into a governed knowledge environment.



O. Operational Agentic Environment & Functional agents

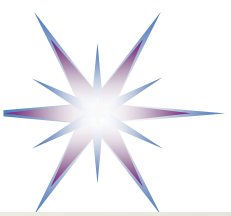
OE. Operational Agentic Environment

The live operational environment where people, events, agents, workflows, models, and enterprise systems interact.

- OE.1 Inputs: humans, events, and tasks
The system receives work from two major sources.
- OE.2 Supervisor or orchestrator agent
The orchestrator is the coordinating component.
- OE.3 Task state and workflow engine
This is one of the most important components for durable tasks.
- OE.4. Enterprise tools and action systems
Agents act through existing enterprise platforms rather than replacing them.

OA. Functional agents

- OA.1 Retrieval and Context Agent
Its purpose is to prevent each operational agent from receiving an uncontrolled or excessively large context.
- OA.2 Software Engineering Agents
Work with repository context, architecture decisions, tests, build systems, and team knowledge.
- OA.3 Customer and Business Operations Agents
- OA.4 System and Infrastructure Agents
- OA.5 Production and Environmental Monitoring Agents
- OA.6 Verification or Critic Agent
The critic agent checks candidate outputs.
- OA.7 Reporting and Explanation Agent
This agent transforms operational results into forms suitable for different audiences.
- OA.8 Knowledge Curator Agent
The curator identifies reusable knowledge arising from operations
- OA.9 Human Escalation and Approval Agent
This component ensures that people are involved.

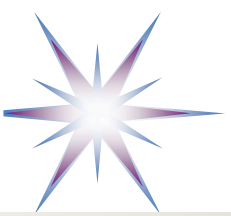


G. Governance and assurance

G. Governance and assurance

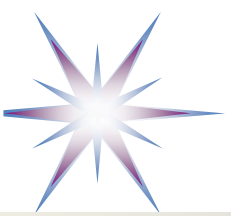
The lower band applies across the whole architecture.

- **G.1 Human-in-the-loop**
Human involvement should be placed at defined responsibility points, not added only when something fails.
- **G.2 Role-based access and security**
Every agent should have: a defined identity; bounded permissions; tool-specific access; separation of duties; controlled handling output formats; escalation; autonomous execution of sensitive context.
- **G.3 Guardrails and policy**
- **G.4 Audit, tracing, and observability**
- **G.5 Quality, risk, and compliance**
Evaluation should cover the full sociotechnical workflow, including human and organisational effects, not only model accuracy.
- **G.6 Metrics**
The architecture proposes a balanced metric
- **G.7 KCEG**
KCEG assesses the complexity and epistemic maturity of produced knowledge
- **G.8 HGLEP**
HGLEP governs the process by which LLM-generated candidate knowledge



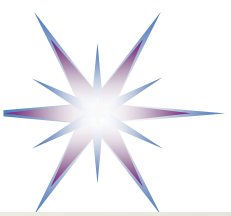
TODO: IASRA Operational Environment and Functional-Agent Design Patterns

- The OE and OA components can be developed following/using a coherent IASRA operational pattern language.
- The important architectural principle is:
 - An agent is responsible for interpretation, planning and recommendation
 - A durable workflow and policy layer remains responsible for state, permissions, approvals, retries and accountable execution
- The functional-agent labels may not necessarily use all suggested LLM services.
A role/service should become a separate agent only when it needs a distinct combination of:
 - knowledge and context
 - tools and permissions
 - evaluation criteria
 - operational owner
 - security boundary
 - scaling or lifecycle requirements
- Several roles can initially be configurations of one agent runtime.



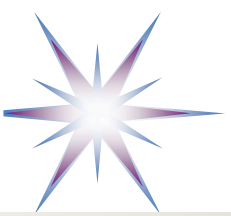
TODO: Implementation Stages and Stacks

- OpenAI Agents SDK
- Google ADE
- Microsoft Agent Framework/Copilot Studio
- Camunda agentic orchestration



Ref: Possible implementation platforms and products

- Enterprise orchestration and governance platform
 - Microsoft / Google / IBM / UiPath / custom
- Business-operation agents
 - Salesforce / ServiceNow / SAP / Oracle
- Software-development agents
 - GitHub / GitLab / custom engineering agents
- Infrastructure and SRE agents
 - Datadog / ServiceNow / cloud-native tools
- Industrial and monitoring agents
 - Siemens / IoT platform / digital twin / custom agents
- Knowledge and context fabric
 - enterprise search + graph DB + vector DB + document repository + operational databases
- Human approval
 - BPM / workflow / case-management mechanisms
- KCEG and HGLEP
 - custom knowledge-assessment and governance layer
- Components communication through APIs, event buses, MCP servers, A2A-style mechanisms, workflow calls, and conventional enterprise integration.



Ref: Practical technologies stack and Implementation stages

Initial implementation

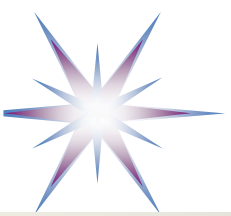
- JSON and JSON Schema for operational messages;
- OpenAPI for services;
- AsyncAPI and CloudEvents for events;
- Relational database for task and reference data;
- Time-series database for monitoring;
- Object store for raw data;
- Git/Markdown for knowledge objects;
- OpenTelemetry for traces and metrics;
- CIM identifiers and metric catalogue.

Second stage

- DCAT catalogue for datasets and services;
- OKF-compatible knowledge packaging;
- SOSA/SSN and QUDT mappings for environmental data;
- SPDX or CycloneDX for software artifacts;
- semantic search over validated documents.

Advanced federation stage

- RDF/OWL representation of selected CIM elements;
- SHACL validation;
- NGSI-LD context broker where digital-twin and subscription use cases justify it;
- PROV-O-inspired provenance model;
- Knowledge graph;
- MCP interfaces for controlled agent access;
- A2A where independently deployed agents must cooperate.

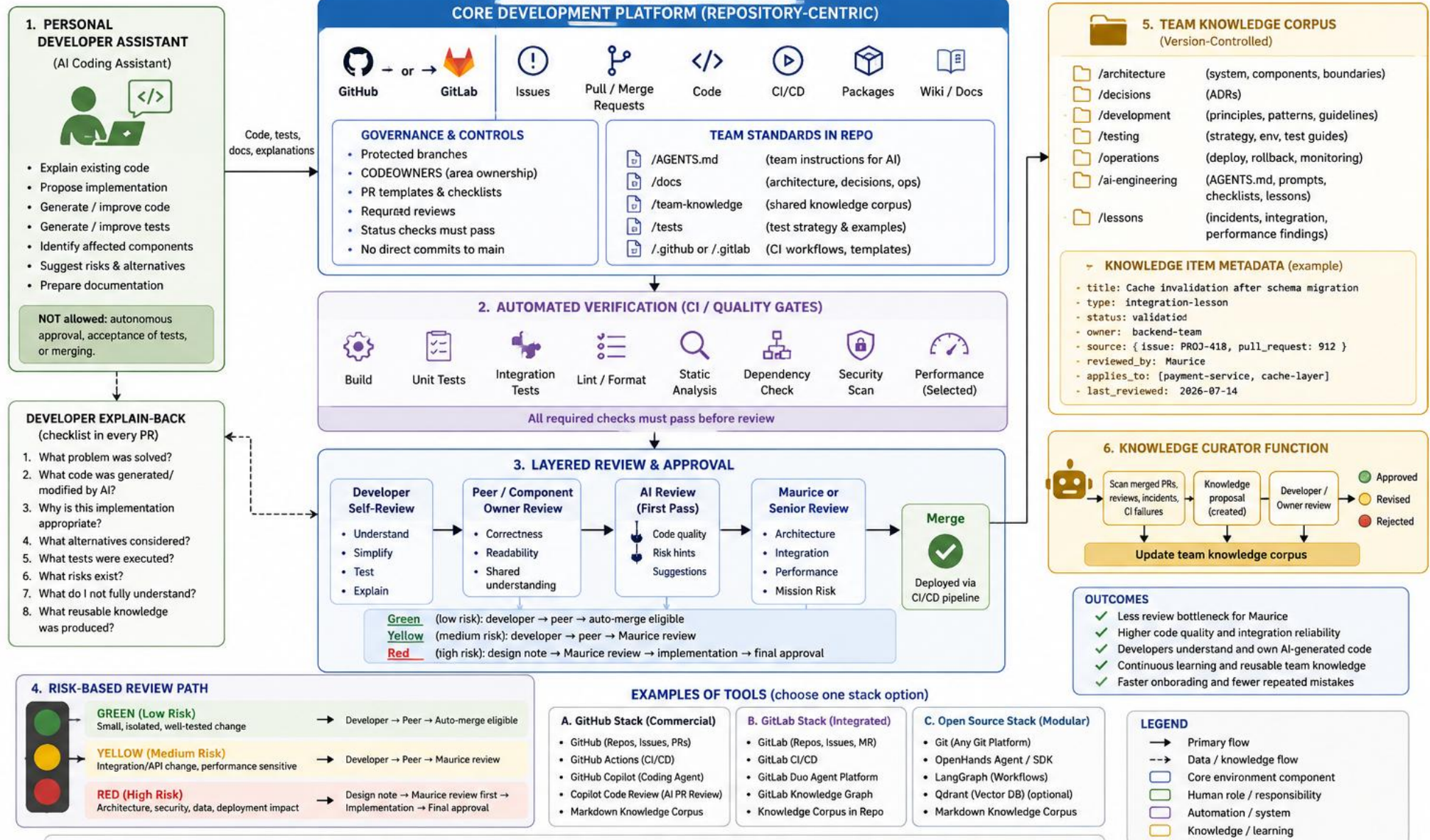


Ref: Trending technologies and Industry Frameworks

Concept	What it really is	Potential benefit	Recommendation for SME
MCP	Agent-to-tool and context protocol	Reusable access to repositories, CI, documentation and internal tools	Plan a limited pilot later
MLCommons	Benchmarks and evaluation practices	Helps compare performance, quality, safety and risk	Use as reference; do not “install”
OKF	Portable knowledge packaging format	Makes team knowledge human- and agent-readable	Prepare compatibility; pilot selectively
CoT	Internal or prompted reasoning process	Can help models solve complex tasks	Do not treat as audit evidence
A2A	Agent-to-agent communication protocol	Interoperability among independent agents	Watch; adopt only when multiple agents require it
OpenAPI	Formal API description	Clear tool contracts, testing and integration	Use now where APIs exist
OpenTelemetry	Tracing, metrics and logging conventions	Visibility into agent calls, tools, cost and failures	Introduce early
SLSA	Software supply-chain assurance framework	Build provenance and stronger artifact integrity	Important for mission-critical work
RAG	Retrieval of selected knowledge for an LLM	Grounds agents in project documentation	Add when the knowledge corpus grows

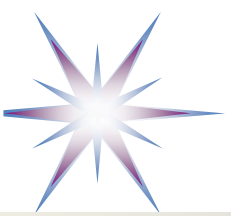
MINIMUM VIABLE AGENTIC DEVELOPMENT ENVIRONMENT

for a Small Software Development Team (10-15 developers)



Case Study 1: Minimum Viable Agentic Development Environment – Main Components

CORE PRINCIPLE: AI produces candidate code. Developer owns understanding. CI provides evidence. Peers create shared understanding. Maurice governs architecture and risk. Knowledge generation turns experience into team assets.



Case Study 1: Minimum Viable Agentic Development Environment – Main Components

A.1 Core repository-centred development platform

GitHub or GitLab should remain the system of record

- The repository workflow performs much of the initial orchestration:

Issue → branch → implementation → PR/MR → CI → review → merge

- Protected branches, required reviews, passing status checks, and code ownership ensure that neither a developer nor an agent can unilaterally place unverified code into a critical branch. GitHub and GitLab both provide these controls, although the exact availability depends on the selected product tier.

A.2 Developer explain-back

Every AI-assisted pull request includes a short declaration.

- This is the principal competence-preservation mechanism. It prevents a developer from acting only as a messenger between the coding agent and Ramen.

1 Personal Developer Assistant

Each developer may use an approved coding assistant

- The assistant produces **candidate engineering artifacts**. It does not approve its own result, accept tests, or merge code.
- The same commercial or open-source coding assistant may be used by all developers, but its behaviour should be guided by common repository instructions. GitHub supports repository-specific custom instructions and code-review instructions; GitLab Duo similarly supports repository-managed review guidance.

2 Automated verification and quality gates

Every proposed change passes through automated checks before substantive human review.

- This layer is deliberately independent of the LLM. It provides objective evidence about the candidate code.
- The most important practical prerequisite is a reproducible test environment.
- An AI assistant may invoke these commands, but developers must also understand and be able to run them.

3. Layered review and approval → See Box →

Review is divided into responsibility levels.

- Developer self-review
- Peer or component-owner review
- AI first-pass review
- Team Leader (Ramen) or senior review

4 Risk-based review path

Changes are classified before or during implementation Green, Yellow, Red.

- For red tasks, Ramen intervenes before the AI generates a large implementation. This is essential for reducing late architectural corrections.

5 Team knowledge corpus

The team knowledge corpus is maintained in version-controlled form close to the code.

- Each important knowledge object should have an owner, status, evidence source, applicable components, and review date.

6 Knowledge Curator function

The Knowledge Curator examines completed development work and asks whether it produced reusable knowledge.

- The curator proposes an update, but a developer, component owner, or Ramen validates it before publication.
- At first, this can be a checklist or scheduled review. Later, it can become a small agent workflow.
- OpenHands provides a software-agent SDK for agents that work with source code, files, and commands, while LangGraph provides persistence and human-interruption mechanisms for workflows that must pause and resume around approval.

3. Layered review and approval

Review is divided into responsibility levels.

Developer self-review

- understand;
- simplify;
- test;
- explain.

Peer or component-owner review

- correctness;
- readability;
- compliance with team practices;
- shared understanding.

AI first-pass review

- possible defects;
- style or quality observations;
- missing tests;

- risk hints.

Team Leader (Ramen) or senior review

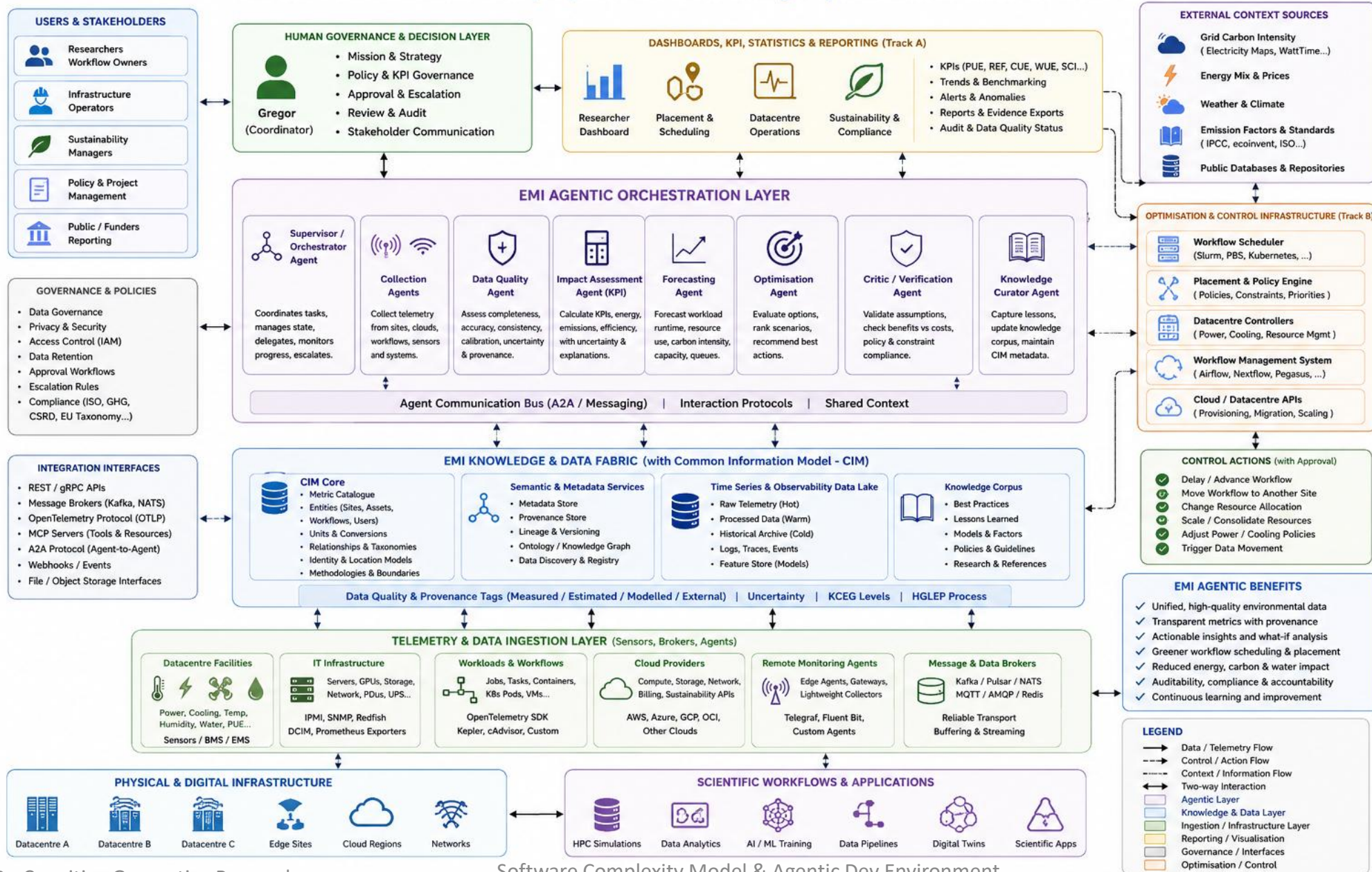
- architecture;
- integration;
- performance;
- security;
- mission-critical risks.

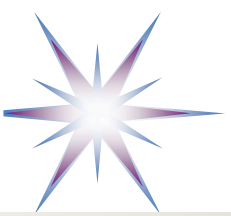
AI code review is already available in products such as GitHub Copilot and GitLab Duo, and both allow review behaviour to be influenced through project instructions. It should be treated as advisory, not as final approval.

Agentic Environmental Monitoring Infrastructure (EMI) Architecture

with Common Information Model (CIM) and Links to Monitoring & Optimisation Infrastructure and Workflow

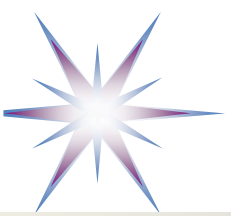
Case Study 2: Agentic Environmental Monitoring Infrastructure (EMI) Architecture





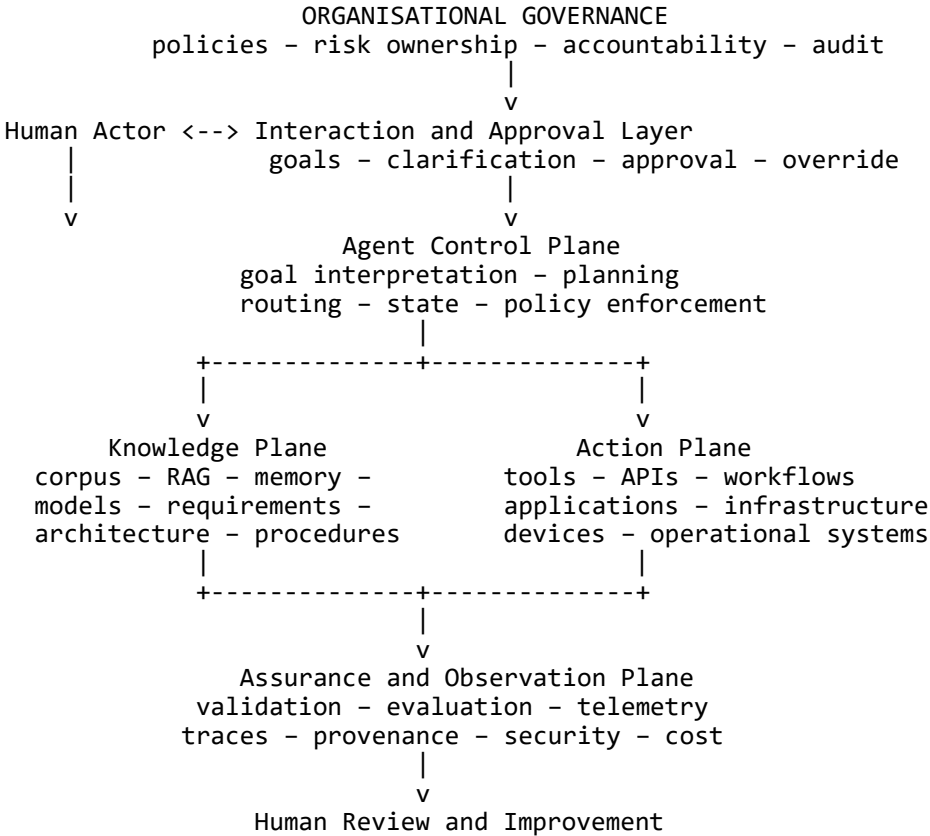
Case Study 2: Agentic Environmental Monitoring Infrastructure (EMI) Architecture - Components

- To add components description



Agentic Systems Design Patterns – Description Elements

Pattern element	Description
Name	Recognisable name used by developers and operators
Context	Organisational and technical situation
Problem	Recurring problem addressed
Forces	Competing requirements, risks and trade-offs
Architecture	Components and interactions
Agent authority	What the agent may observe, propose, decide or execute
Human role	Human responsibilities before, during and after execution
Controls	Guardrails, policies, approvals and constraints
Evidence	Logs, traces, decisions, evaluations and provenance
Failure handling	Abort, retry, rollback, escalation or safe state
Prerequisites	Competences and infrastructure required
Applicable adoption stage	Where the pattern enters the organisational progression





Architecture Patterns (1)



Pattern A1 — Human-Led Agentic Assistant

Context

A practitioner uses an LLM or agent through an interactive interface for explanation, drafting, analysis or code generation.

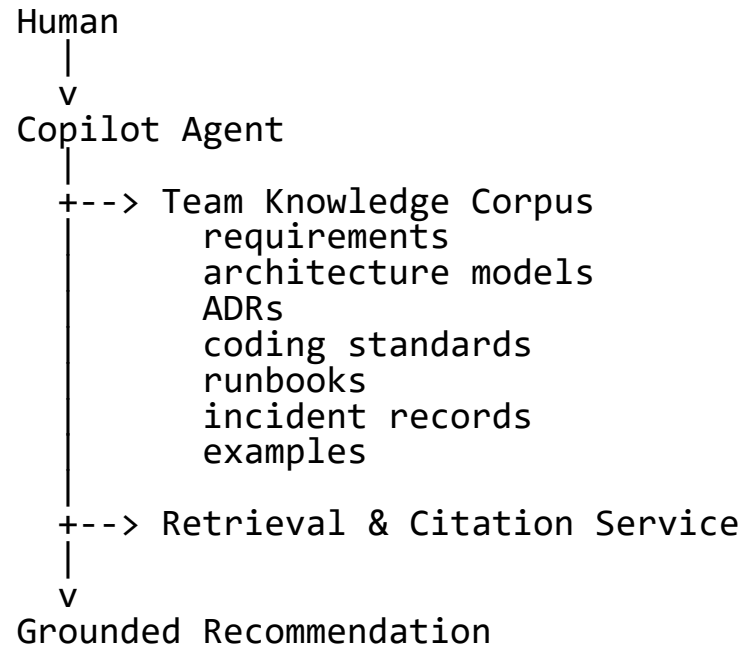
Architecture

Human -> Prompt/Task -> Assistant
Agent -> Candidate Output -> Human

Pattern A2 — Knowledge-Grounded Team Copilot

A generic assistant does not reliably follow team conventions, architecture decisions or operational procedures.

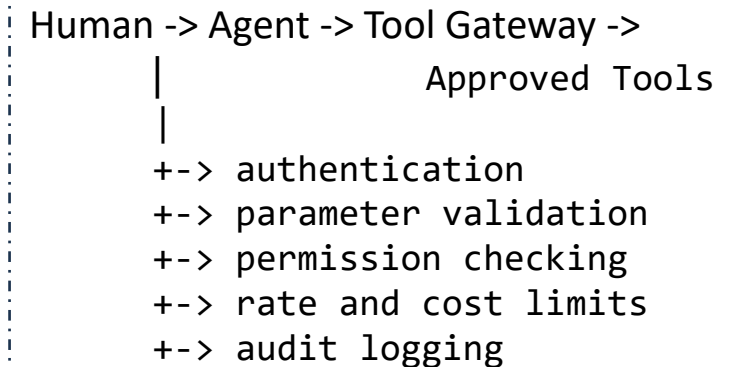
Architecture



Pattern A3 — Bounded Tool-Using Agent

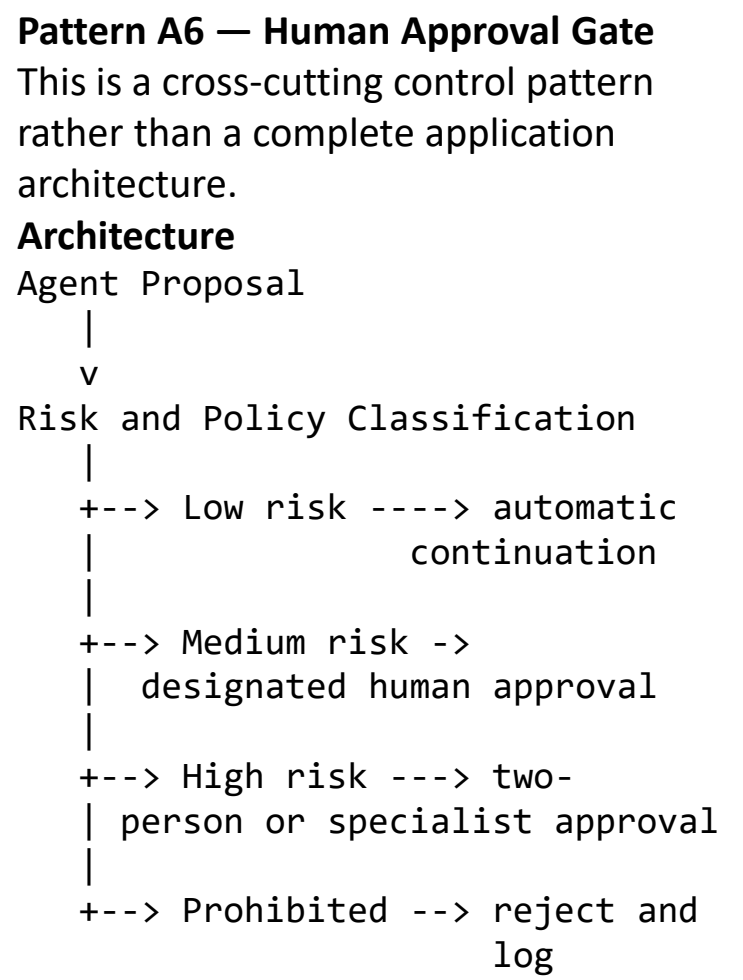
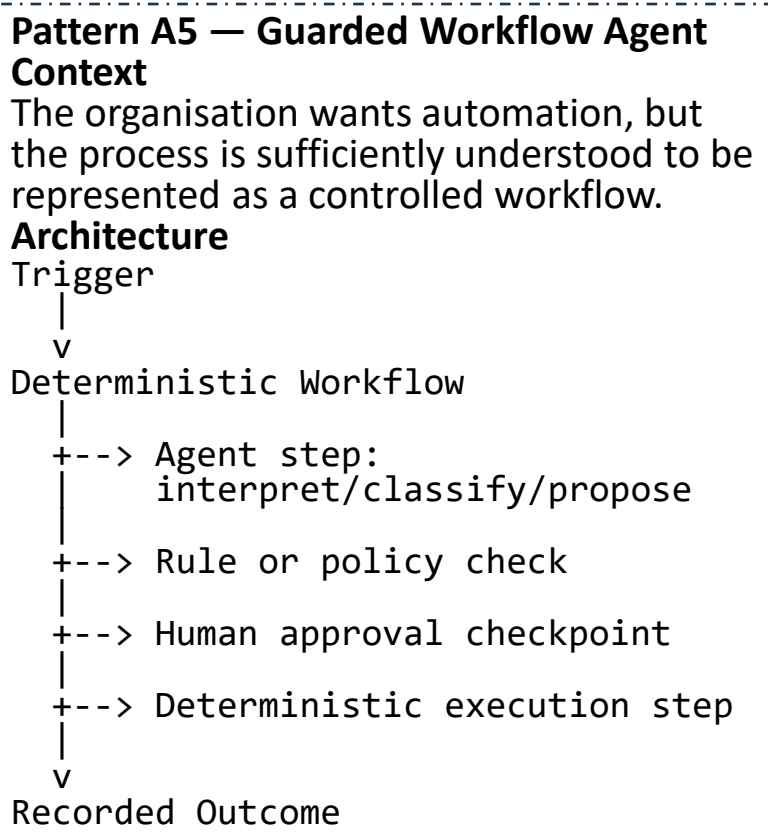
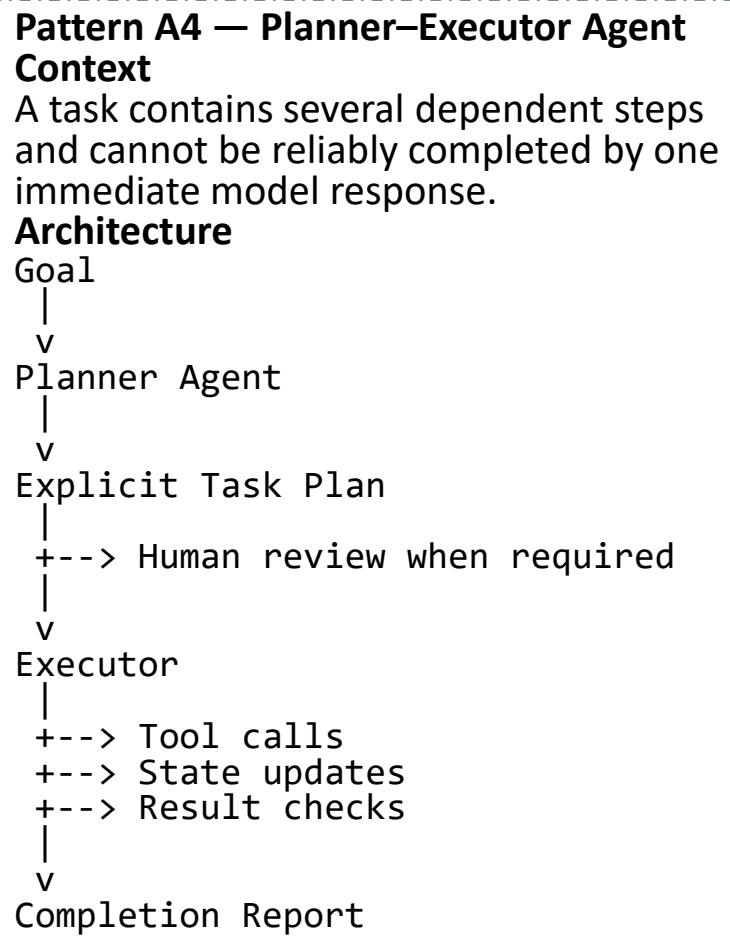
The agent needs to inspect repositories, query systems, run tests or perform other limited actions.

Architecture





Architecture Patterns (2)

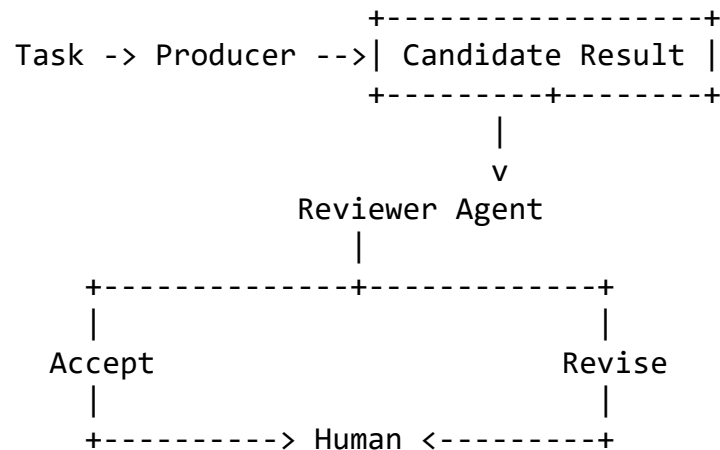


Pattern A7 — Independent Reviewer or Critic Agent

Context

The primary agent may overlook defects in its own output.

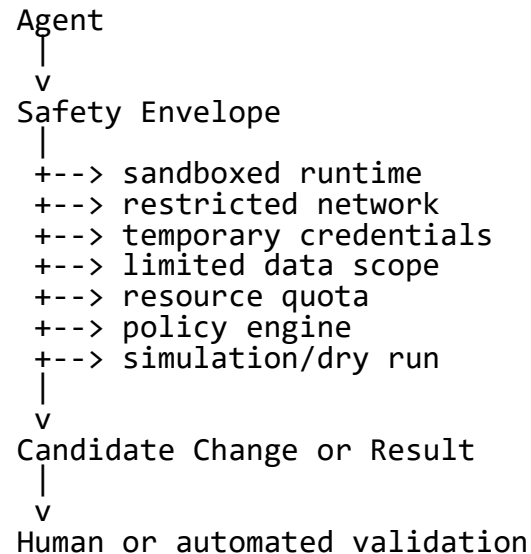
Architecture



Pattern A8 — Sandbox and Safety Envelope Context

The agent needs execution capability, but mistakes must not propagate to production systems.

Architecture

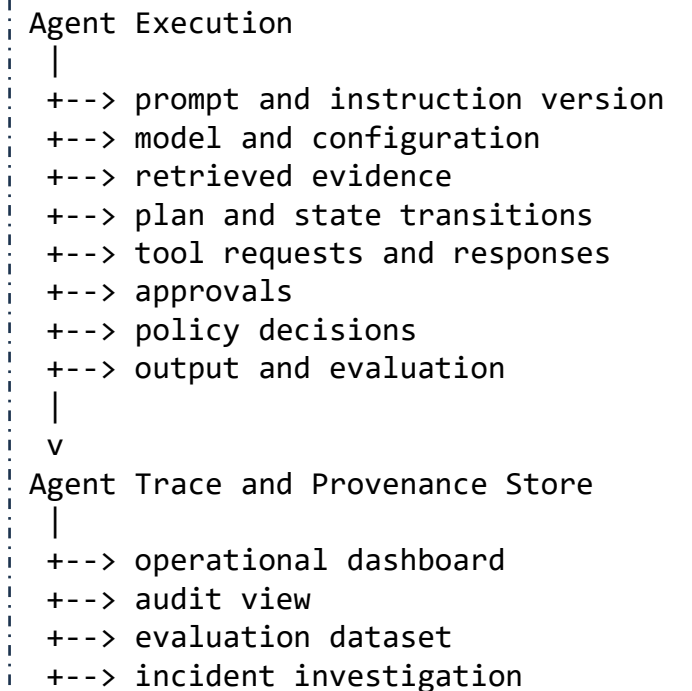


Pattern A9 — Observable and Reproducible Agent

Context

A team cannot improve, debug or govern an agent if it cannot reconstruct what the agent did.

Architecture



Pattern A10 — Event-Driven Observer-Advisor Agent

Context

The agent continuously monitors events but should not directly control operations.

Architecture

Systems and Sensors



Event and Telemetry Bus



Observer Agent



- +--> anomaly detection
- +--> contextual interpretation
- +--> impact assessment
- +--> recommended response



Operator or Operational Officer

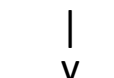
Pattern A11 — Supervised Closed-Loop Control Agent

Context

The organisation has sufficient confidence to let the agent make bounded operational changes.

Architecture

Observe -> Analyse -> Plan -> Policy Check -> Act



+----- Verify <- Telemetry <- System



- +--> Human override
- +--> rollback
- +--> safe state

Pattern A12 — Supervisor-Specialist Multi-Agent Team

Context

One agent accumulates too many tools, instructions and responsibilities.

Architecture

Human Goal



Supervisor / Coordinator Agent



- +--> Requirements Agent
- +--> Architecture Agent
- +--> Development Agent
- +--> Testing Agent
- +--> Security Agent
- +--> Operations Agent



Integration and Conflict Resolution



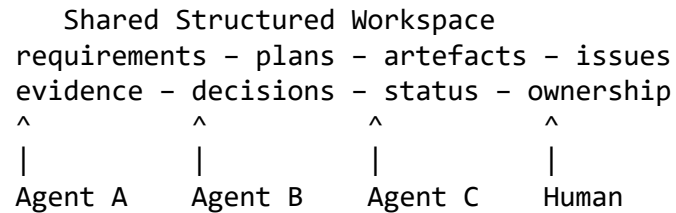
Human Review

Pattern A13 — Shared Workspace or Blackboard

Context

Specialist agents need to collaborate without repeatedly passing entire conversational histories.

Architecture

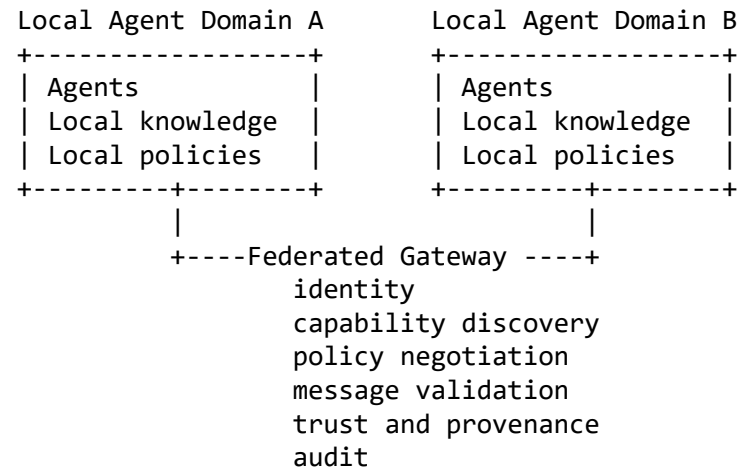


Pattern A14 — Federated Agent Gateway

Context

Different departments or organisations operate their own agents, knowledge sources and security domains.

Architecture



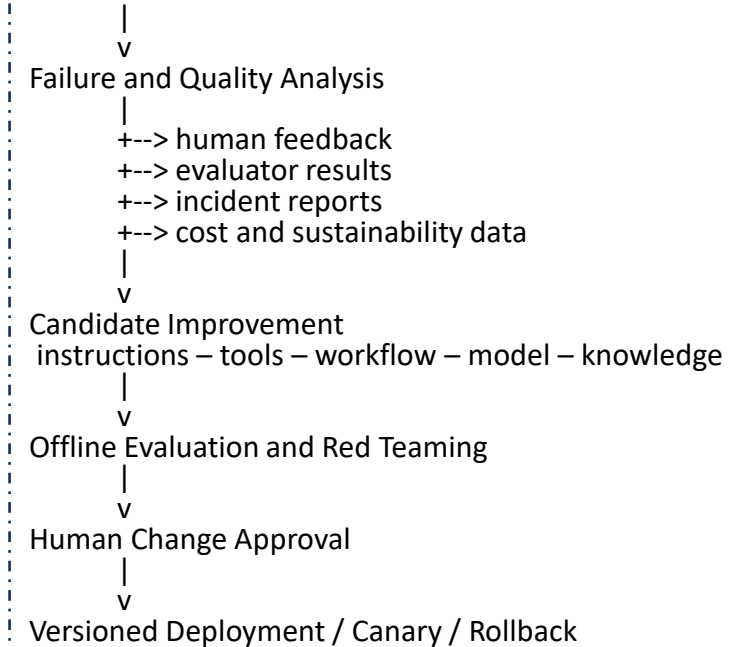
Pattern A15 — Governed Agent Improvement Loop

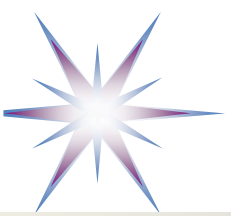
Context

Agent performance must improve using operational evidence without allowing uncontrolled self-modification.

Architecture

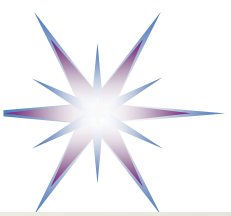
Runtime Traces and Outcomes





Agentic Knowledge Management and Agentic Knowledge Architecture (AKA)

- Agentic Knowledge components
- Agentic Knowledge Architecture (AKA) and Team Agentic DevOps Knowledge System (TAKS)

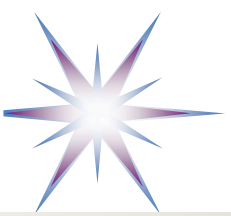


Data, Information, Knowledge – Knowledge types

- Level 1. Physical Reality
 - Starting point of all data, information, knowledge: objects, events, processes, people
- Level 2. Data
 - Data are symbolic observations about reality.
- Level 3. Information
 - Information is interpreted data placed into context.
- Level 4. Structured Information
 - Information becomes structured by relations: ERP, CRM, ontology, workflow, knowledge graph
- Level 5. Operational Knowledge
 - First level that can be qualified as knowledge: Provides a basis for what should be done: rules. protocols
- Level 6. Conceptual Knowledge
 - Basic/foundational concepts why things exist: architecture, scientific theory, domain models
- Level 7. Methodological Knowledge
 - Methodology to create knowledge: scientific & engineering methods
- Level 8. Epistemic Knowledge
 - Basis for knowledge quality and validation, scientific reasoning and peer review
- Level 9. Frontier Knowledge
 - Generative Knowledge

The current Knowledge hierarchy can be applied equally to:

- business information systems,
- enterprise knowledge management,
- AI agents,
- scientific research,
- LLMs,
- human cognition.



Knowledge Used in ADE and Existing Tools

Knowledge form	Typical storage	Role in agentic systems
Prompts / instructions	Markdown, YAML, JSON, code	Defines agent role, behaviour, constraints
Tool knowledge	JSON Schema, OpenAPI, MCP tool definitions	Defines what actions the agent can call
Document knowledge	PDFs, HTML, Markdown, wiki pages	Source material for RAG
Vector knowledge	Vector DB / embedding index	Semantic retrieval
Structured data	SQL, data warehouse, APIs	Authoritative business facts
Ontological knowledge	RDF/OWL, property graphs, taxonomy stores	Concepts, entities, relationships
Process knowledge	BPMN, workflow engine, state machine	Business process state and transitions
Episodic memory	Session store, event log, conversation state	What happened in this task or case
Audit knowledge	Logs, traces, tool-call history	Accountability and evaluation
Model knowledge	LLM weights, fine-tuned models	General linguistic and reasoning capacity

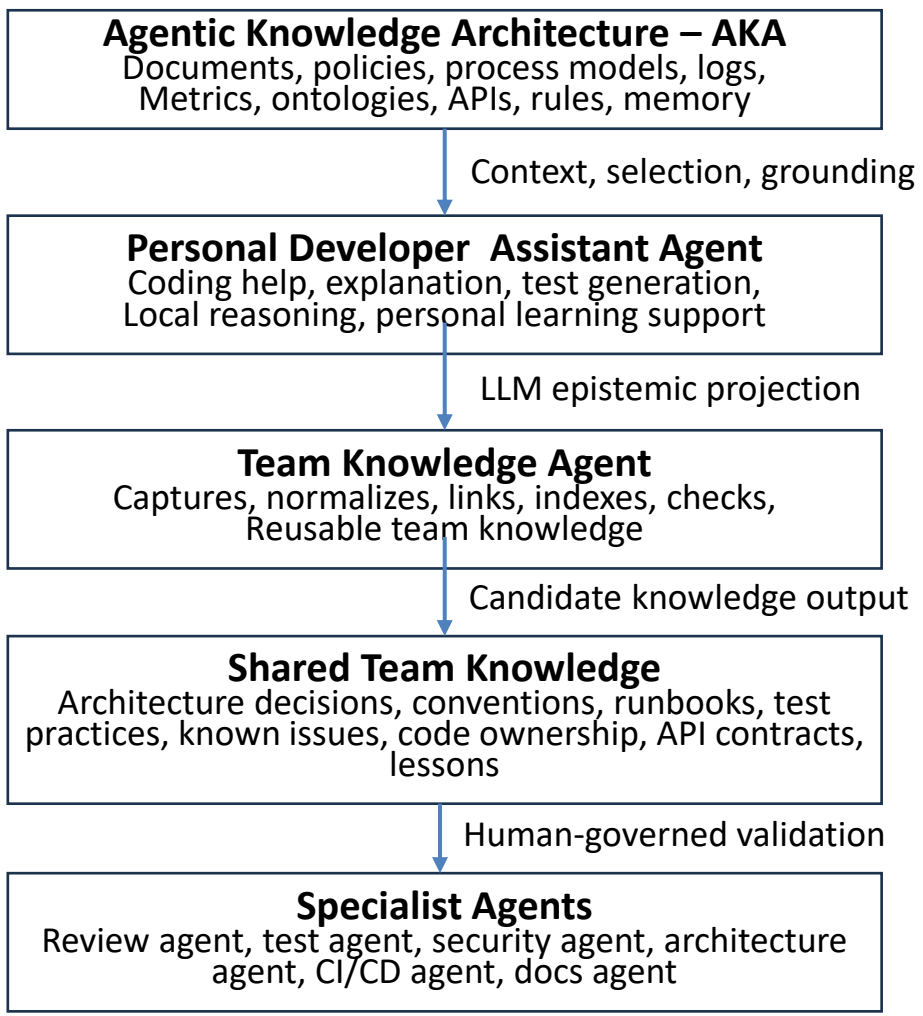
Knowledge presentation and management tools

- RAG based knowledge presentation
- Knowledge Graph
- Workflow/state-based
- Tool/Schema based

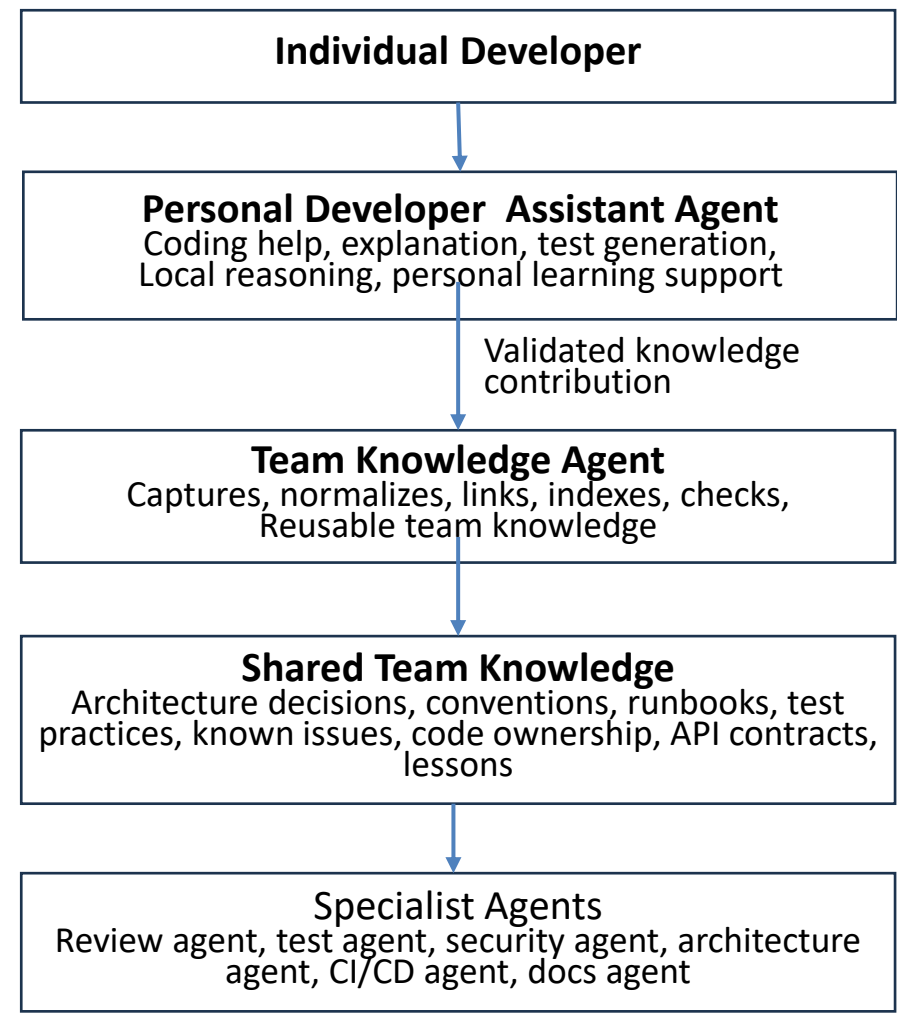
- Open Knowledge Format (OKF) by Google (June 2026)
 - OKF bundle
 - concept_1.md
 - concept_2.md
 - index.md
 - log.md
 - linked markdown documents



Agentic Knowledge Architecture (AKA) & Team Agentic Knowledge System (TAKS)

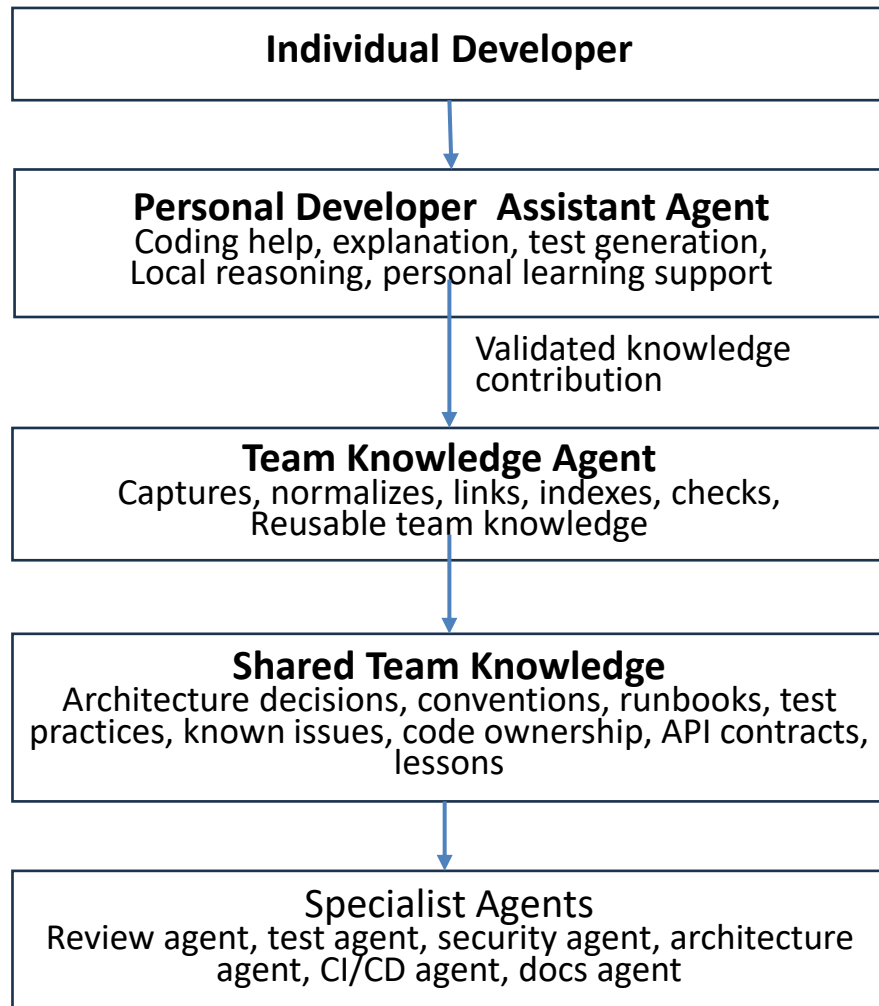


- **Loop 1 — PR knowledge capture**
When a pull request is opened, an agent asks
- **Loop 2 — Review learning loop**
A review agent collects repeated review comments
- **Loop 3 — Incident-to-knowledge loop**
After a production incident
- **Loop 4 — Sprint retrospective loop**
The team agent analyses
- **Loop 5 — Onboarding loop**
A new developer's personal assistant can ask the team knowledge agent

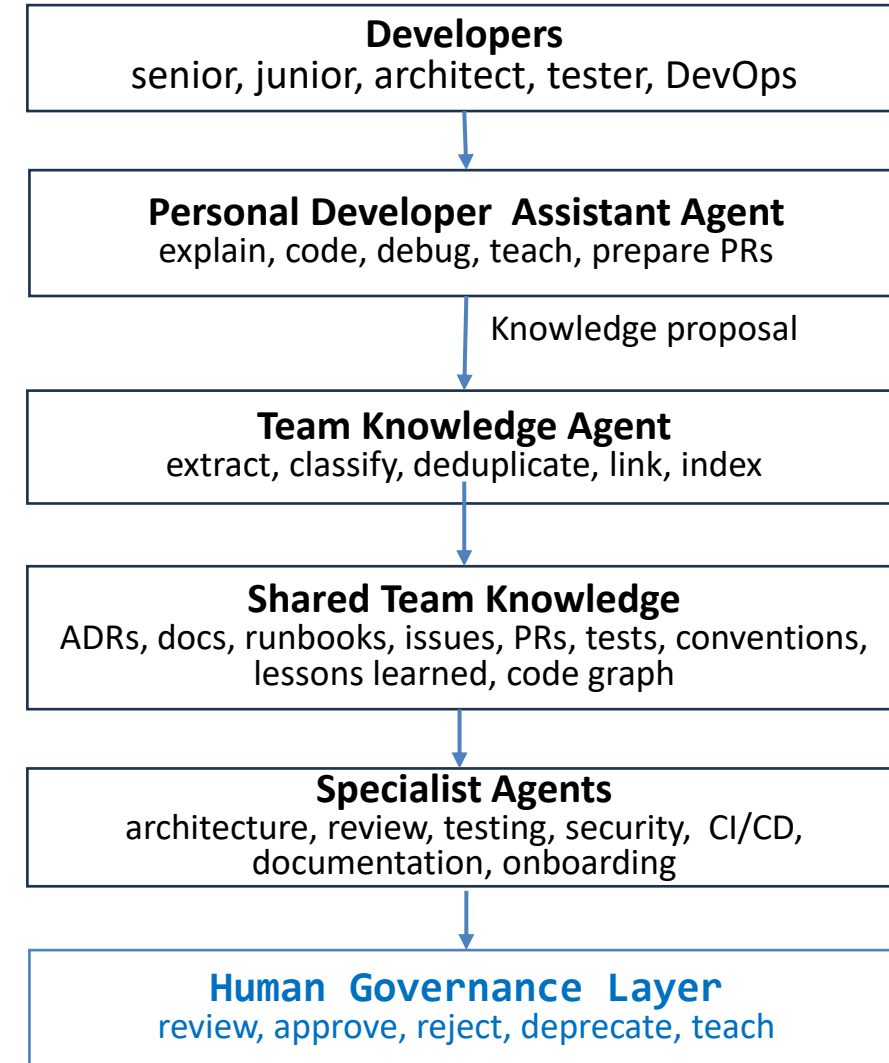


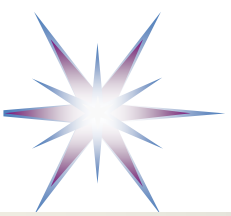


Extending Team Agentic Knowledge System (TAKS) to Team Agentic DevOps Knowledge System (TADoKS)



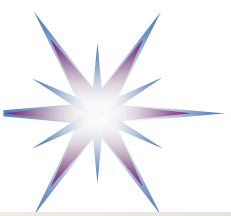
- **Loop 1 — PR knowledge capture**
When a pull request is opened, an agent asks
- **Loop 2 — Review learning loop**
A review agent collects repeated review comments
- **Loop 3 — Incident-to-knowledge loop**
After a production incident
- **Loop 4 — Sprint retrospective loop**
The team agent analyses
- **Loop 5 — Onboarding loop**
A new developer's personal assistant can ask the team knowledge agent



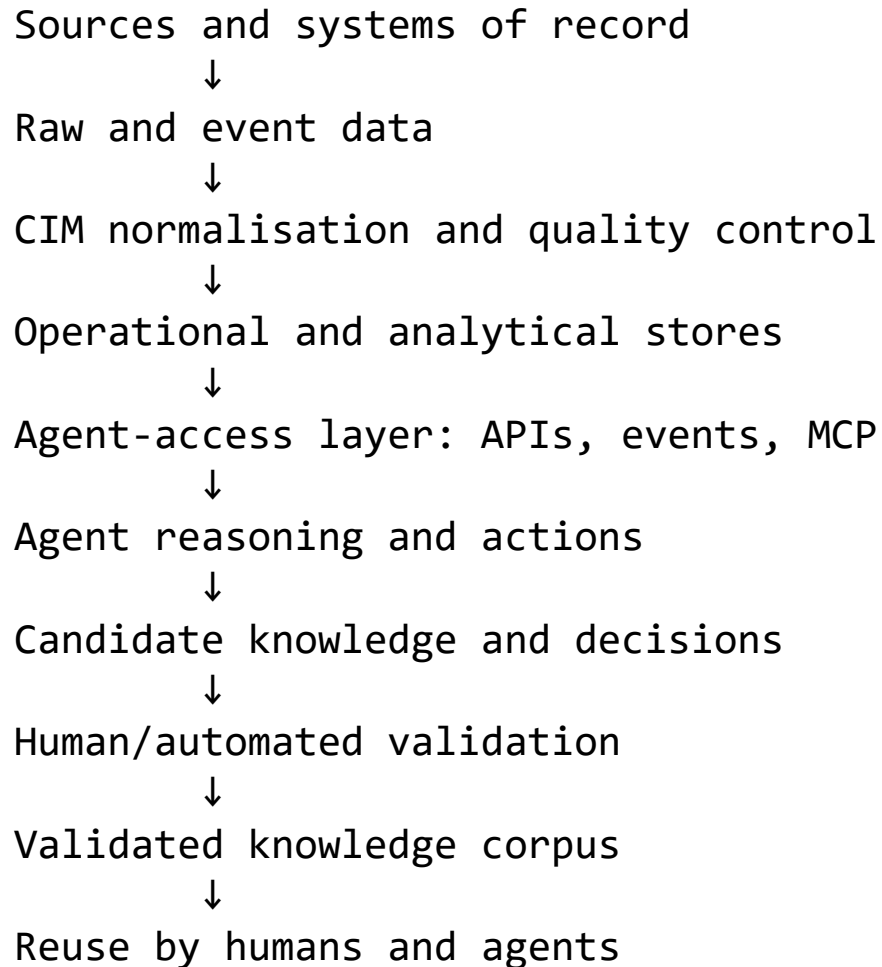


Applying KCEG and HGLEP to Agentic Systems Interaction and Team Knowledge Management

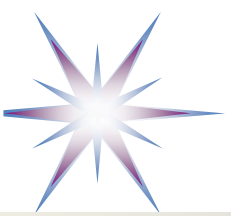
- Linking and aligning existing frameworks and tools for Agents' interaction
 - MCP – Model Context Protocol (K1 – K4/KC1-KC2) (Anthropic, 25 November 2025)
 - OKF – Open Knowledge Format (Google, 12 June 2026)
- Introducing Agentic Knowledge Architecture (AKA)
 - What knowledge is available to the agentic system?
 - Knowledge supply, acquisition, storage, retrieval, context, lifecycle
- MCP layer and main LLM - IBM
 - LLM mirroring for knowledge sharing
 - API Gateway – iPaaS?



AKA/TAKS and IASRA Data Architecture Elements



- AKA and TAKS should coordinate several specialised stores through a common information model, provenance envelope, controlled interfaces, and explicit knowledge lifecycle.
- The CIM defines **what the data means**.
 - OpenAPI, AsyncAPI, CloudEvents, MCP, and A2A define **how systems and agents exchange it**.
 - PROV-O, DCAT, SOSA/SSN, QUDT, SPDX, and related standards provide reusable domain structures.
 - KCEG and HGLEP define **how generated information becomes trusted knowledge**.
- TODO: Clarify the place and role for
 - OKS (Open Knowledge Protocol)
 - CIM (Common Information Model) to extend to Knowledge elements/packages



Metaphoric Graphics to support importance of Architecture Knowledge and Structured Information Perception

ONE BIG MENTAL BAG



MANY MENTAL POCKETS

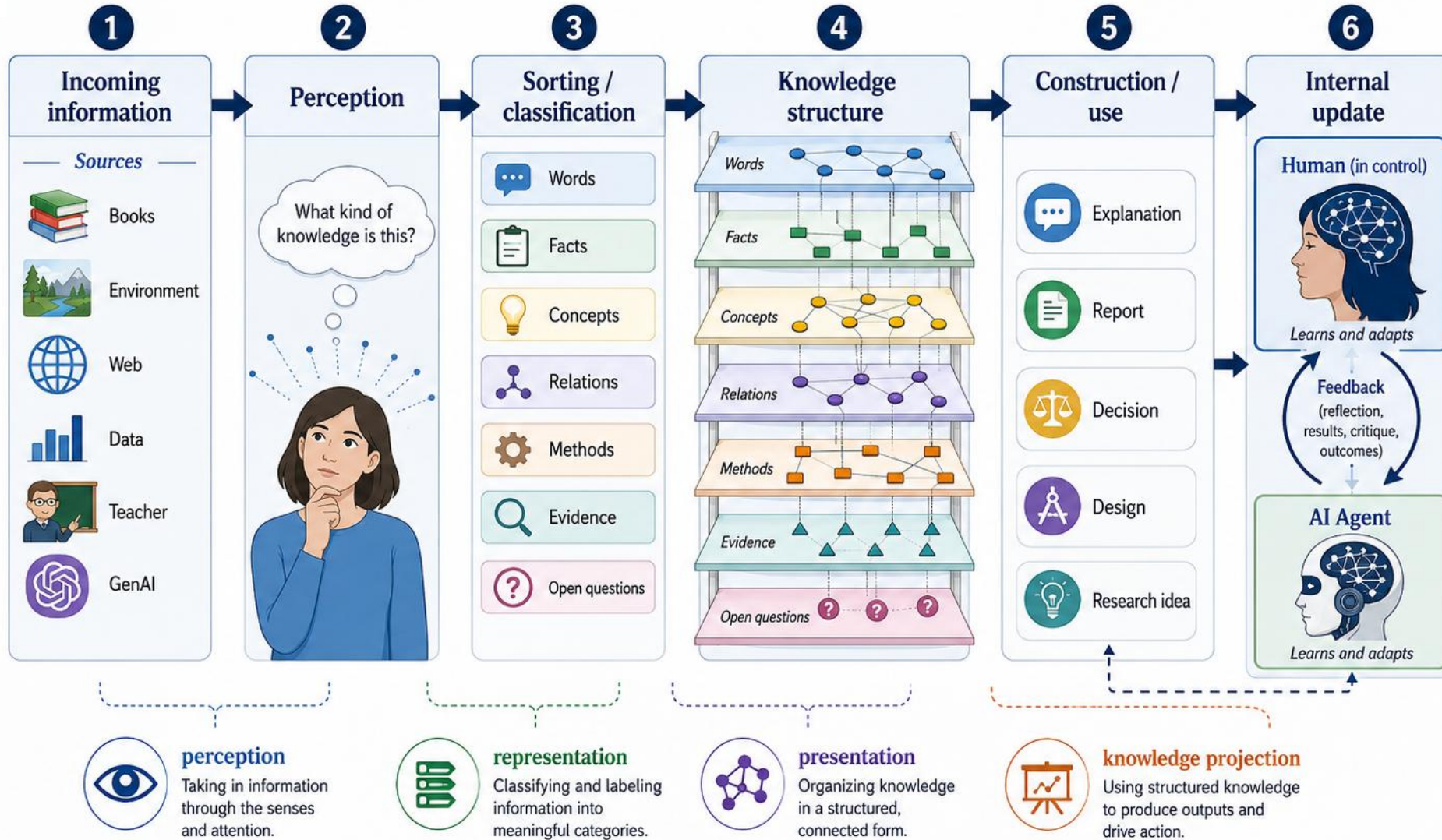


Using KCEG
(Knowledge
Complexity &
Effective
Guidance) Model

* For structured
Perception,
Learning and
building
Knowledge corpus

KCEG helps you sort information while it arrives.

KCEG as a Lens for Human-Agent Interaction



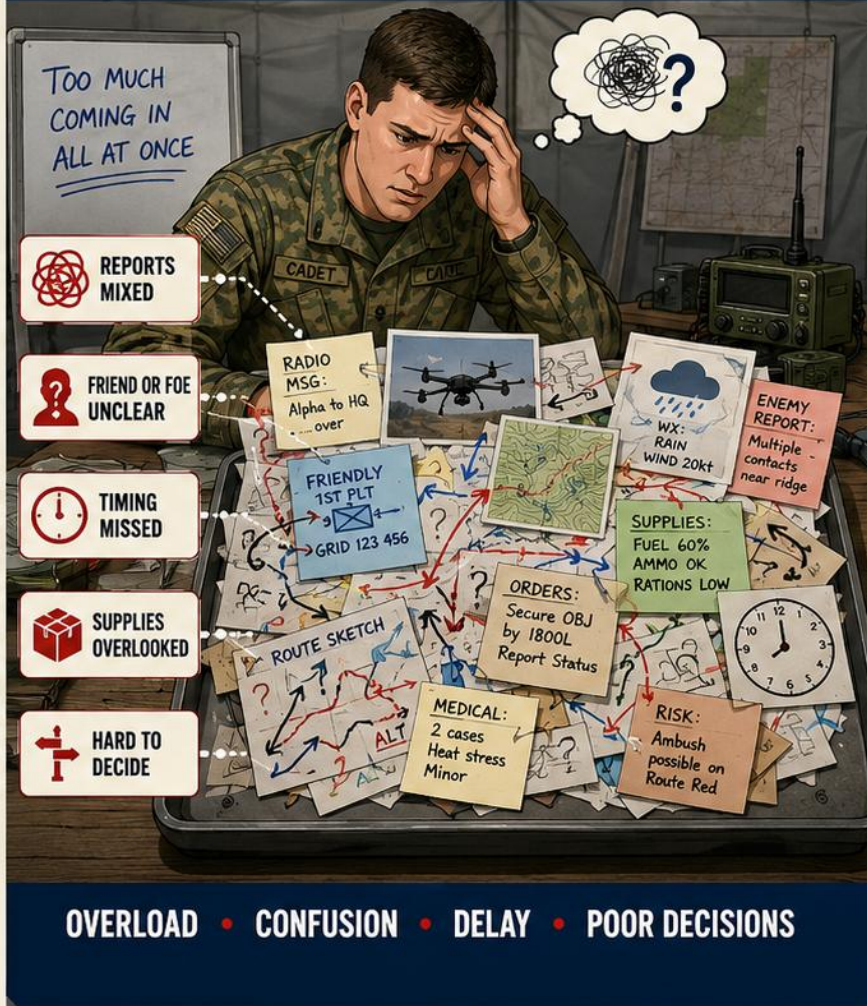
Using KCEG (Knowledge Complexity & Effective Guidance) Model

- * For structured Perception, Learning and building Knowledge corpus
- * Main instrument for scientific research and professional development & productivity
- * Framework to build Human – Agent interaction



The human remains responsible for structure, judgment, and update.

ONE BIG INFORMATION PILE



Too MUCH COMING IN ALL AT ONCE

REPORTS MIXED

FRIEND OR FOE UNCLEAR

TIMING MISSED

SUPPLIES OVERLOOKED

HARD TO DECIDE

RADIO MSG: Alpha to HQ over

FRIENDLY 1ST PLT GRID 123 456

WX: RAIN WIND 20kt

ENEMY REPORT: Multiple contacts near ridge

SUPPLIES: FUEL 60% AMMO OK RATIONS LOW

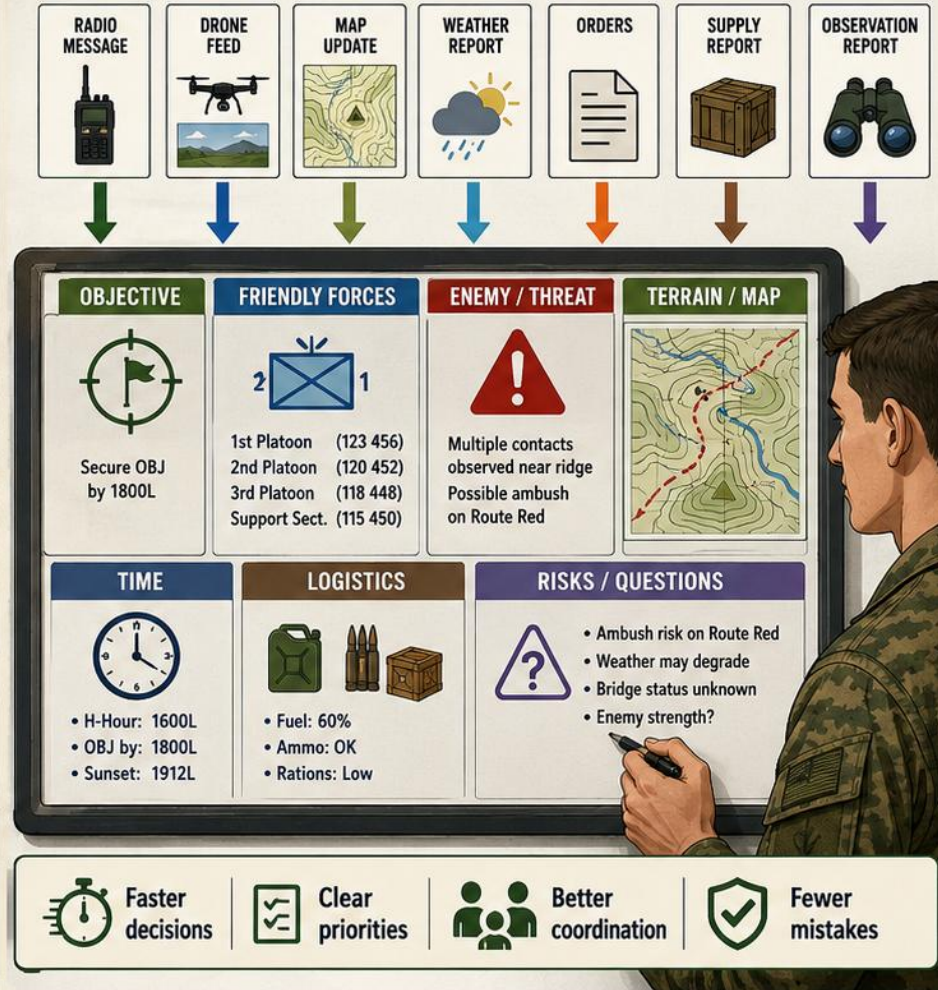
ORDERS: Secure OBJ by 1800L Report Status

MEDICAL: 2 cases Heat stress Minor

RISK: Ambush possible on Route Red

OVERLOAD • CONFUSION • DELAY • POOR DECISIONS

STRUCTURED SITUATIONAL AWARENESS



RADIO MESSAGE

DRONE FEED

MAP UPDATE

WEATHER REPORT

ORDERS

SUPPLY REPORT

OBSERVATION REPORT

OBJECTIVE	FRIENDLY FORCES	ENEMY / THREAT	TERRAIN / MAP
Secure OBJ by 1800L	1st Platoon (123 456) 2nd Platoon (120 452) 3rd Platoon (118 448) Support Sect. (115 450)	Multiple contacts observed near ridge Possible ambush on Route Red	
TIME	LOGISTICS	RISKS / QUESTIONS	
• H-Hour: 1600L • OBJ by: 1800L • Sunset: 1912L	• Fuel: 60% • Ammo: OK • Rations: Low	• Ambush risk on Route Red • Weather may degrade • Bridge status unknown • Enemy strength?	

Faster decisions

Clear priorities

Better coordination

Fewer mistakes

Using KCEG (Knowledge Complexity & Effective Guidance) Model

* For structured information processing for mission critical situation management and dynamic adaptation & reaction



Sort information while it arrives — this is how your mind builds knowledge.

In the field, structured information becomes situational understanding.



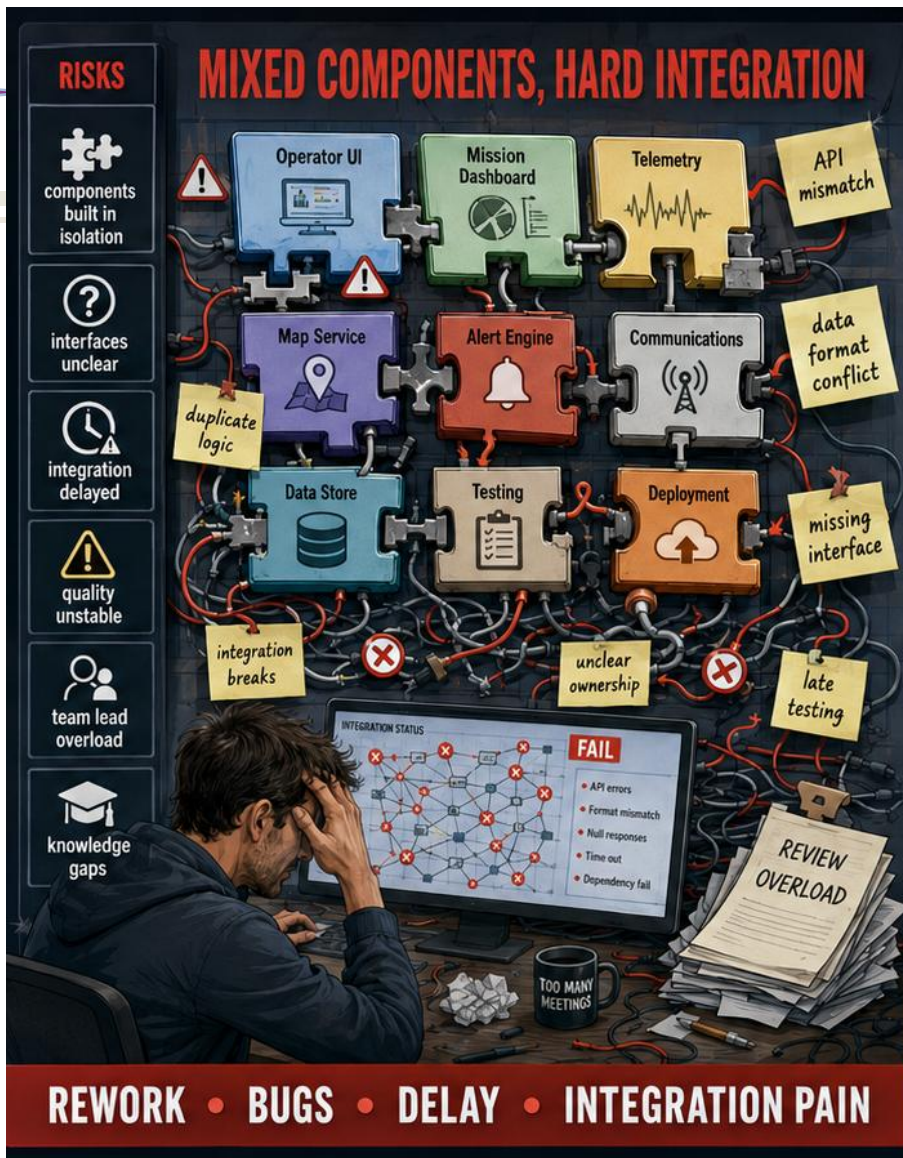
Importance of
architecture design
principles
knowledge & SCEG

- For coordinated multi-component system development
- To embed architecture compliance and smooth integration

MIXED COMPONENTS, HARD INTEGRATION

RISKS

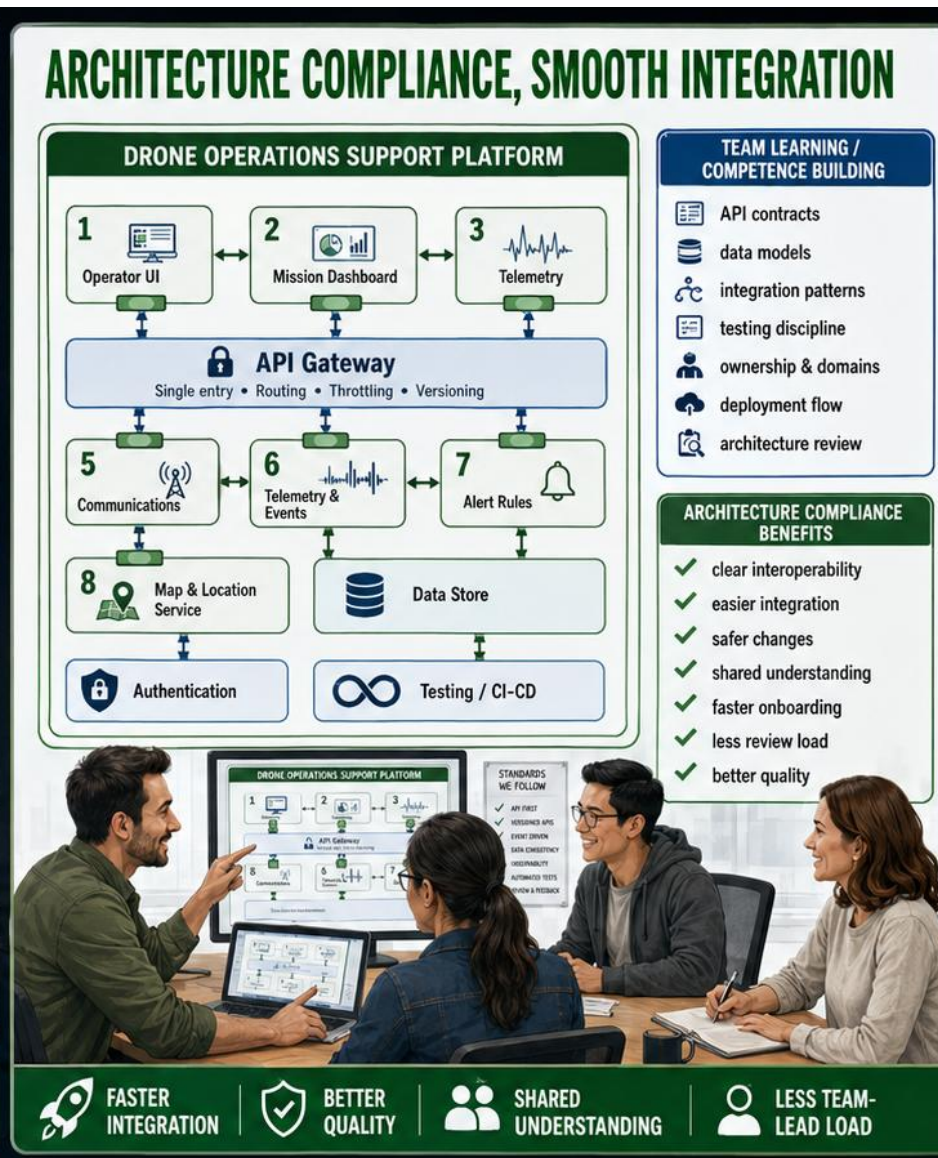
- components built in isolation
- interfaces unclear
- integration delayed
- quality unstable
- team lead overload
- knowledge gaps



REWORK • BUGS • DELAY • INTEGRATION PAIN

ARCHITECTURE COMPLIANCE, SMOOTH INTEGRATION

DRONE OPERATIONS SUPPORT PLATFORM



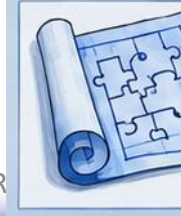
TEAM LEARNING / COMPETENCE BUILDING

- API contracts
- data models
- integration patterns
- testing discipline
- ownership & domains
- deployment flow
- architecture review

ARCHITECTURE COMPLIANCE BENEFITS

- ✓ clear interoperability
- ✓ easier integration
- ✓ safer changes
- ✓ shared understanding
- ✓ faster onboarding
- ✓ less review load
- ✓ better quality

FASTER INTEGRATION | BETTER QUALITY | SHARED UNDERSTANDING | LESS TEAM-LEAD LOAD

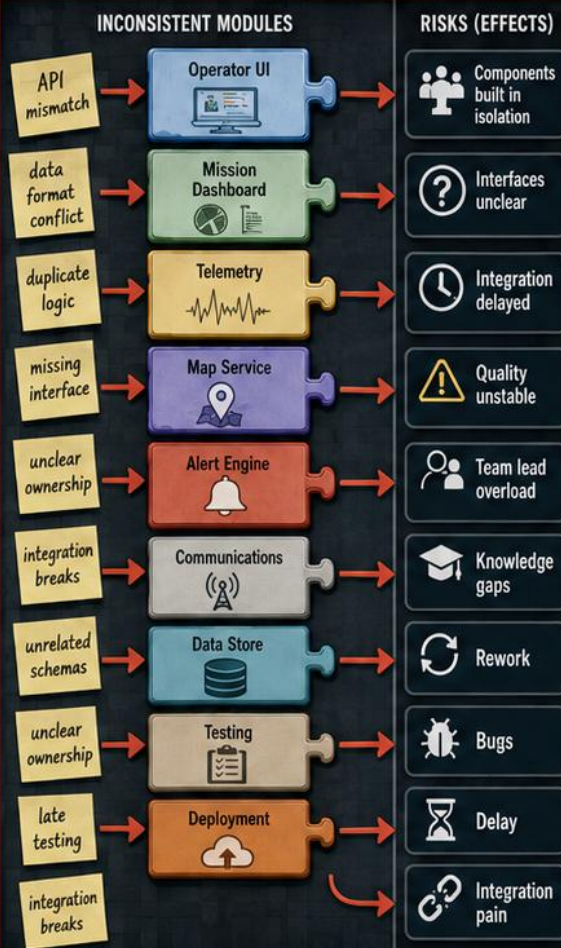


Learn architecture while you build – this is how your team turns separate components into one dependable system.

When components follow shared design principles, interoperability and integration become easier.

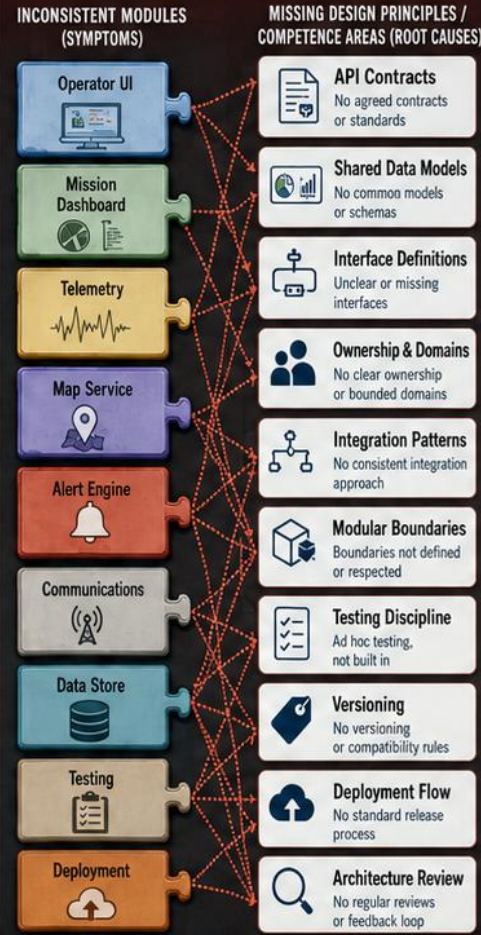


MODULE INCONSISTENCIES CREATE RISKS



INCONSISTENCIES → RISKS → REWORK, BUGS, DELAY, INTEGRATION PAIN

MODULE INCONSISTENCIES REVEAL MISSING DESIGN PRINCIPLES



CAUSE: MISSING PRINCIPLES → EFFECT: INCONSISTENCIES & RISKS

ARCHITECTURE COMPLIANCE, SMOOTH INTEGRATION



Importance of architecture design principles knowledge & SCEG

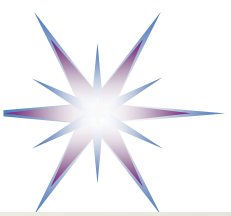
- For coordinated multi-component system development
- To embed architecture compliance and smooth integration
- Modules inconsistencies reveal missing design principles



Learn architecture while you build – this is how your team turns separate components into one dependable system.

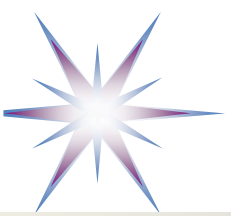
When components follow shared design principles, interoperability and integration become easier.





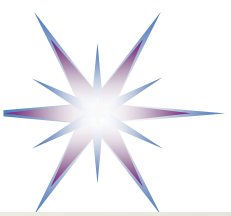
Q&A

- Q: Can LLMs create knowledge? How LLM assistance can facilitate learning and education?
- A: Simple answer – LLM doesn't create knowledge the same way as a researcher potentially gains knowledge horizon when interacting with LLM Assistant
 - LLM as learning assistant can provide interactive support:
 - expose uncertainty
 - distinguish fact, inference, hypothesis, analogy, and speculation
 - help map concepts and claims
 - reveal gaps and contradictions
 - support verification
 - allow expert researchers to govern the process



References

- Anthropic article
 - 2026 Agentic Coding Trends Report. How coding agents are reshaping software development. Anthropic, claude.ai, 2026 <https://resources.anthropic.com/hubfs/2026%20Agentic%20Coding%20Trends%20Report.pdf>
- NIST AI Risk Management Framework (AI RMF), Jan 2023
- [*NIST-AI-600-1, Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*](#), July 26, 2024
- [*Concept note for an AI RMF Profile on Trustworthy AI in Critical Infrastructure*](#), April 7, 2026.



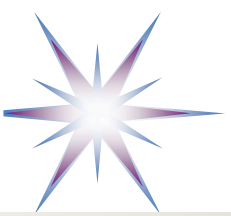
Software Development Lifecycle with Agentic Coding

2026 Agentic Coding Trends Report. How coding agents are reshaping software development. Anthropic, claude.ai, 2026

<https://resources.anthropic.com/hubfs/2026%20Agentic%20Coding%20Trends%20Report.pdf>

Three prediction categories:

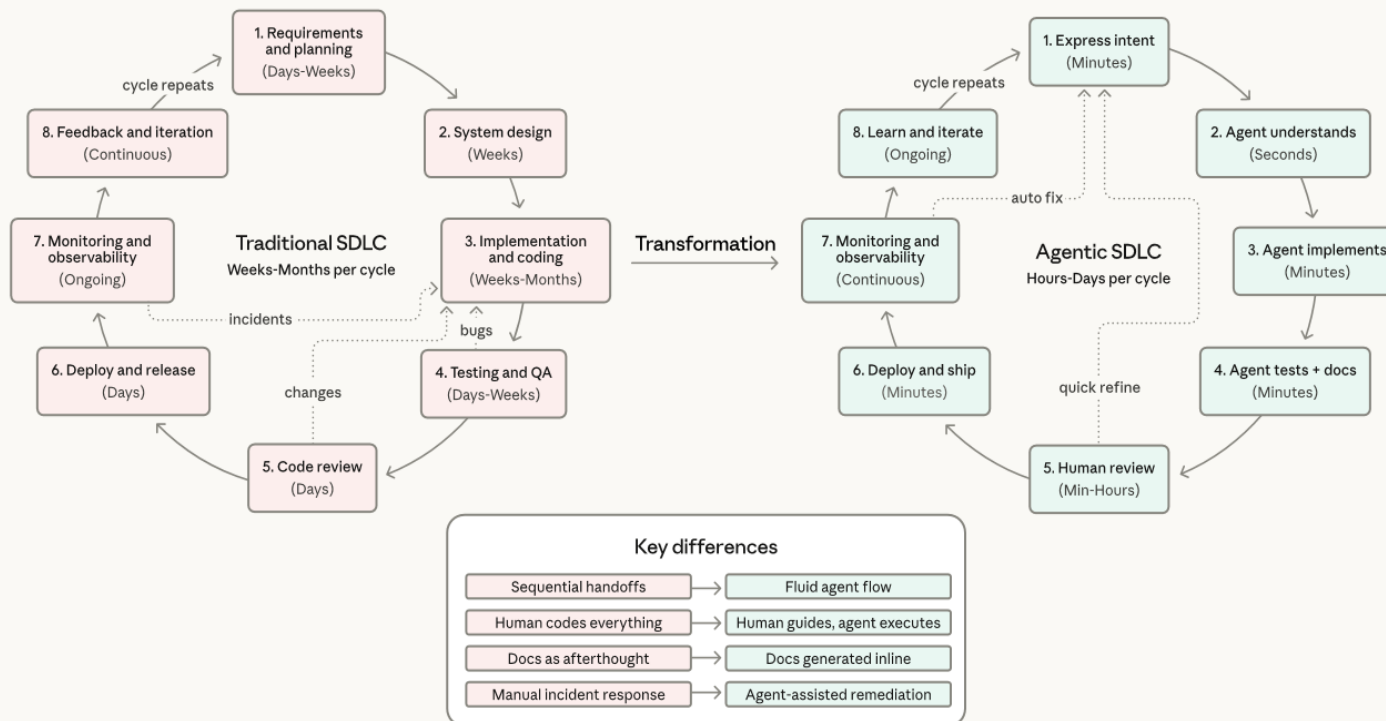
- Foundation trends that we believe will reshape how development work happens
- Capability trends that look to expand what agents can accomplish
- Impact trends that we anticipate will affect business outcomes and organizational structures.



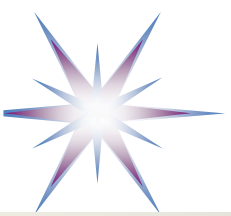
Foundation trends: The tectonic shift

Trend 1 - The software development lifecycle changes dramatically

Software development life cycle: Before and after agentic coding tools

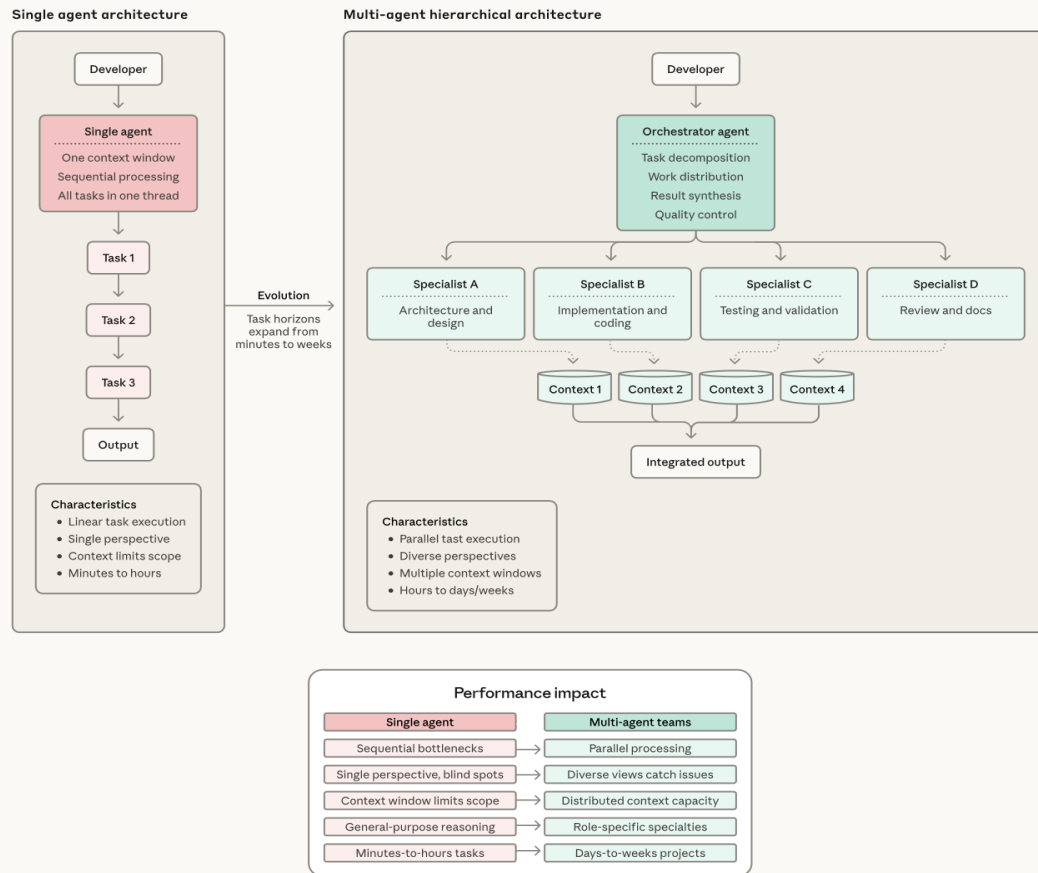


- **Evolution of abstraction:** Most of the tactical work of writing, debugging, and maintaining code shifts to AI while engineers focus on higher-level work like architecture, system design, and strategic decisions about what to build.
- **Engineering role transformation:** Building software used to mean primarily writing code, although software engineering roles always involved many other skills. Now, being a software engineer increasingly means orchestrating agents that write code, evaluating their output, providing strategic direction, and ensuring the system as a whole solves the right problems correctly.
- **Expedited onboarding to dynamic project staffing:** Traditional timelines for onboarding to a new codebase or project will collapse from weeks to hours, changing how companies think about talent deployment and project resourcing.
- **Collaborative reality:** Tighter feedback loops and faster learning.



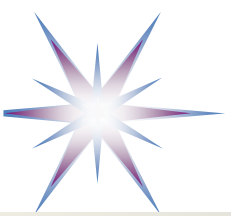
Capability trends: What agents can do

Coding agent architectures: From single agents to coordinated teams



- Trend 2 Single agents evolve into coordinated teams
 - **Multi-agent systems replace single-agent workflows:** Organizations adopt multi-agent workflows that maximize performance gains through parallel reasoning across separate context windows.

Single-agent workflows process tasks sequentially through one context window. Multi-agent architectures use an orchestrator to coordinate specialized agents working in parallel—each



Capability trends: What agents can do

- **Trend 3 Long-running agents build complete systems**

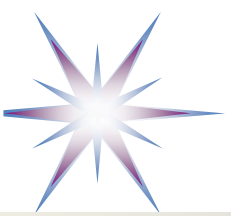
- **Task horizons expand from minutes to days or weeks:** Agents evolve from handling discrete tasks that complete in minutes to working autonomously for extended periods, building and testing entire applications and systems with periodic human checkpoints.
- **Agents handle the messy reality of software development:** Long-running agents plan, iterate, and refine across dozens of work sessions, adapting to discoveries, recovering from failures, and maintaining coherent state throughout complex projects.
- **Economics of software development change:** When agents can work autonomously for extended periods, formerly non-viable projects become feasible. Technical debt that accumulated for years because no one had time to address it gets systematically eliminated by agents working through backlogs.
- **Path to market accelerates:** Entrepreneurs use agents to go from ideas to deployed applications in days instead of months.

- **Trend 5 Agentic coding expands to new surfaces and users**

- **Language barriers disappear:** Support expands to less-common and legacy languages like COBOL, Fortran, and domain-specific languages, enabling maintenance of legacy systems and removing adoption barriers for specialized use cases.
- **Coding democratizes beyond engineering:** New form factors and interfaces open up agentic coding to non-traditional developers in fields like cybersecurity, operations, design, and data science. Tools like [Cowork](#), designed for non-developers to automate file and task management, signal this shift is already underway.

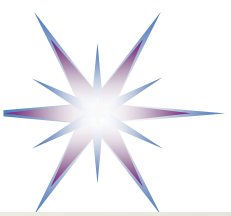
- **Trend 4: Human oversight scales through intelligent collaboration**

- **Agentic quality control becomes standard:** Organizations use AI agents to review large-scale AI-generated output, analyzing code for security vulnerabilities, architectural consistency, and quality issues that would overwhelm human capacity.
- **Agents learn when to ask for help:** Rather than blindly attempting every task, sophisticated agents recognize situations requiring human judgment, flagging areas of uncertainty and elevating decisions with potential business impact.
- **Human oversight shifts from reviewing everything to reviewing what matters:** Teams maintain quality and velocity simultaneously by building intelligent systems that handle routine verification while escalating genuinely novel situations, boundary cases, and strategic decisions for human input.

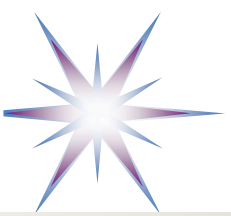


Impact trends: What agents may change in 2026

- **Trend 6 Productivity gains reshape software development economics**
 - Three multipliers drive acceleration: Agent capabilities, orchestration improvements, and better use of human experience compound to create stepfunction improvements rather than linear gains as each enables the others.
 - Timeline compression changes project viability: Development that once took weeks now takes days, making previously unviable projects feasible and enabling organizations to respond to market opportunities more quickly.
 - Economics of software development shift: Total cost of ownership decreases as agents augment engineer capacity, project timelines shorten, and faster time to value improves return on investment.
Productivity through output volume, not just speed
- **Trend 7 Non-technical use cases expand across organizations**
 - Coding capabilities democratize beyond engineering: Non-technical teams across sales, marketing, legal, and operations gain the ability to automate workflows and build tools with little or no engineering intervention or coding expertise.
 - Domain experts implement solutions directly: The hands-on experts who understand problems deeply gain confidence in using agents to initiate solutions themselves, removing the bottleneck of filing a ticket and then waiting for development teams.
 - Productivity gains extend across entire organizations: Problems not worth engineering time get solved, experimental workflows become trivial to attempt, and manual processes get automated.
- **Trend 8 Agentic coding improves security defenses— but also offensive uses**
 - **Security knowledge becomes democratized:** With improved agents, any engineer can become a security engineer capable of delivering in-depth security reviews, hardening, and monitoring. Engineers will still need to consider security and consult with specialists, but it will become easier to build hardened and secure applications.
 - **Threat actors scale attacks:** While agents will benefit defensive uses, they will also benefit offensive uses too. In order to defend against this dual-use technology, it will become more important for engineers to build in security from the start.
 - **Agentic cyber defense systems rise:** Automated agentic systems enable security responses at machine speed, automating detection and response to match the pace of autonomous threats.



Related information



Knowledge Complexity Aspects

- General, linked to knowledge presentation
 - Vocabulary and semantics describe **how knowledge is expressed** (in words and terms).
 - Ontology and structure describe **how it is organised**.
 - Inference and methodology describe **how it works and how it is produced**.
 - **Epistemic measures describe how reliable, novel, uncertain, or contested it is.**
- For humans, complexity is strongly tied to prior knowledge, cognitive load, and research judgment.
- For LLMs, complexity is tied to representation, grounding, context, retrieval, reasoning depth, and verifiability.
- The measure cannot be a single score but a **complexity vector** across these dimensions.